

SCM: Autonomous Lifelong Learning for Language Agents via Sleep-Stage Memory Consolidation

Technical Report

SCM Research Team
blobopera@proton.me

May 2026

Abstract

We present SCM (Sleep-Consolidated Memory), the first open-source memory architecture for language agents that learns autonomously during user idle time. Existing memory systems are stateless when not in use: they store facts when told and retrieve them when asked, but never consolidate, abstract, or grow on their own. SCM closes this gap with a complete biological memory lifecycle: bounded working memory, selective encoding with attention-gated intensity classification, event-structured semantic binding with Hebbian weight updates, spreading-activation retrieval with context-gated hypothesis ranking, dual-mode sleep consolidation, and contradiction-safe version lineage — and on top of these, a Phase 7 autonomous-learning layer that activates while the user is away. Phase 7 contributes six new subsystems: an idle-aware background daemon that fires sleep cycles when sessions go quiet (M1); a cross-session memory pool that lets consolidation span days rather than single conversations (M2); a schema extractor that emits typed pattern concepts during REM (M3); a wake-up summary endpoint that surfaces what the agent learned overnight (M4); a curiosity engine with pluggable knowledge sources, including a static dictionary, a local-docs scanner, and an opt-in LLM source (M5); and a power-state-aware lifecycle policy with crash-safe state persistence (M6). All locale-specific knowledge — regex patterns, stoplists, narrative templates, paraphrase rules — lives in an externalized JSON file rather than in code. We validate the system through a brutal testing methodology that augments unit tests with persona-driven multi-day simulations, ad-

versarial inputs, failure-mode injection, and scale runs. The harness uncovered four real bugs that 170+ unit tests missed (schema-ID instability, spreading-activation seed purging, dead cross-session pool in single-user case, and superseded-concept retrieval leakage); each was root-caused, fixed, and verified by an additional brutal scenario. Real-LLM validation on Ollama (free, local) and DeepSeek (cloud, paid) confirms the system functions end-to-end with non-stub extractors. We release the complete system, the brutal harness, and all benchmark artifacts as reproducible open-source infrastructure.

Contents

1	Introduction	2
1.1	Design Principles	3
2	Related Work	4
2.1	Memory-Augmented Language Models	4
2.2	Autonomous and Lifelong Agents	4
2.3	Continual Learning and Forgetting	5
2.4	Neuroscientific Foundations . . .	5
3	System Architecture	6
3.1	Data Model	6
3.2	Deployment Tiers	6
4	Phase 1: Selective Encoding	9
4.1	AttentionGate	9
4.1.1	Saliency Model	9
4.1.2	Grasp Score	9
4.2	Integration with the Encoding Pipeline	10

5	Phase 2: Event Structuring and Associative Binding	10	11	Evaluation	22
5.1	Event Compiler	11	11.1	Component-Level Performance . .	22
5.2	Association Binder	11	11.2	Sleep Gain	22
5.2.1	Intra-Event Binding . . .	11	11.3	Baseline Stress Comparison . . .	23
5.2.2	Event-to-Anchor Binding	11	11.4	Long-Horizon Behavior	23
5.2.3	Edge Management	11	11.5	Phase 6 Human-Memory Suite .	23
5.3	Predicate Inference	12	11.6	Regression Safety	23
6	Phase 3: Associative Retrieval	12	11.7	LoCoMo: External Long-Conversation Memory Benchmark	24
6.1	Spreading Activation Retriever .	12	11.8	LoCoMo++: Memory Workload Sensitivity	24
6.2	Hypothesis Ranker	13	11.9	Phase 6 Architecture Fixes . . .	26
7	Phase 4: Dual-Mode Sleep Kernel	13	11.10	Brutal Testing: Persona-Driven Multi-Day Simulation	27
7.1	Sleep Trigger	13	11.11	Bug Discovery via the Brutal Harness	28
7.2	MicroSleep	14	11.12	Real-LLM Validation	29
7.3	DeepSleep	14	12	Discussion	30
8	Phase 5: Forgetting Dynamics and Contradiction Versioning	15	13	Limitations	30
8.1	ForgettingDynamics	15	14	Conclusion	32
8.1.1	Retention Scoring	15			
8.1.2	Trace-Specific Decay . . .	15			
8.1.3	State Transitions	16			
8.2	Contradiction Versioning	16			
8.3	Forgetting Dynamics Integration with Sleep	16			
9	Phase 7: Autonomous Idle-Time Learning	17			
9.1	M1: Idle-Aware Background Daemon	17			
9.2	M2: Cross-Session Memory Pool	17			
9.3	M3: Schema Extraction in REM	18			
9.4	M4: Wake-Up Summary	19			
9.5	M5: Curiosity Engine	19			
9.6	M6: Lifecycle Policy	19			
9.7	De-Hardcoding via LinguisticResources	20			
9.8	Temporal Structure	20			
10	Implementation and Infrastructure	21			
10.1	Persistence Architecture	21			
10.2	API Surface	21			
10.3	Deterministic Demo and Smoke Testing	22			

1 Introduction

Most production language agents do not learn after deployment. They retrieve, they answer, they generate—but the moment a conversation ends, no abstraction, no consolidation, no growth happens before the next one begins. The agent that wakes up tomorrow is exactly the agent that went to sleep last night. This is not how biological memory works, and it is not what users expect when they imagine a personal assistant that “gets to know them.” Humans do not run their agents twenty-four hours a day; the gaps between sessions are precisely the windows in which a useful agent should be doing the work of becoming better. Existing memory systems are silent in those windows.

Three architectural families have addressed the related but narrower problem of *storage*. Context-window approaches Liu et al. [2023] extend token budgets through compression and efficient attention, but suffer from the “lost-in-the-middle” phenomenon and provide no cross-session persistence. Retrieval-augmented generation (RAG) systems Lewis et al. [2020] embed and retrieve by similarity, but grow without bound and offer no importance prioritization, consolidation, or intentional forgetting. Tiered architectures like MemGPT Packer et al. [2023] borrow OS metaphors to page data between fast and slow tiers, but lack the biological processes—sleep-dependent consolidation, adaptive pruning, contradiction resolution—that make human memory functional. Personalized memory layers like Mem0 Chhikara et al. [2025] extract facts in real time but remain *awake-only reactive systems*: they do nothing during the long stretches when the user is away. Cache-management work like SleepGate Xie [2024] borrows the sleep metaphor for KV eviction without creating semantic memory at all.

The human memory system is fundamentally different. Working memory holds approximately seven items in a limited-capacity buffer that forces selective attention Miller [1956]. Long-term memory stores semantic and episodic information in associative networks whose topology reflects co-occurrence Tulving [2002], with

cue-driven rather than ranked retrieval. Sleep is not idle: during NREM, hippocampal replay strengthens cortical connections; during REM, novel associations form and memories integrate Rasch and Born [2013]. Forgetting is active—weak synapses are pruned to preserve signal-to-noise ratios as formalized by the synaptic homeostasis hypothesis Tononi and Cirelli [2003]. The brain detects contradictions and maintains versioned representations with temporal validity rather than blindly overwriting prior beliefs.

This paper presents SCM (Sleep-Consolidated Memory), an open-source memory architecture that implements computational analogs of all these processes within a single coherent runtime, and—critically—adds an autonomous-learning layer that closes the gap between session-bounded retrieval and continuous lifelong learning. The wake-phase substrate (Phases 1–6) provides bounded working memory, attention-gated selective encoding, event-structured semantic binding with Hebbian updates, spreading-activation retrieval with context-gated hypothesis ranking, dual-mode (NREM/REM) sleep consolidation, adaptive value-based forgetting with a salience floor, and contradiction-safe versioning. Phase 7 then activates while the user is away: an idle-aware background daemon (M1) fires sleep cycles when sessions go quiet, a cross-session memory pool (M2) lets consolidation span days rather than single conversations, a schema extractor (M3) emits typed pattern concepts during REM, a wake-up summary endpoint (M4) surfaces what the agent learned overnight, a curiosity engine (M5) with pluggable knowledge sources (static dictionary, local-docs scanner, opt-in LLM source) lets the agent autonomously fill its own knowledge gaps, and a power-state-aware lifecycle policy (M6) with crash-safe state persistence ensures the daemon respects battery, CPU, and thermal constraints. The complete system is gated behind a single feature flag (`HME_ENABLED`, the Human Memory Engine) that enables staged deployment from baseline SCM to the full lifelong-learning agent.

Contributions. This paper makes five primary contributions:

1. **Integrated lifecycle architecture.** SCM unifies selective encoding, event binding, spreading-activation retrieval, dual-mode sleep, adaptive forgetting, and contradiction versioning within a single open-source runtime. Prior work addresses individual aspects—EWC for parameter-level catastrophic forgetting Kirkpatrick et al. [2017], WSCL for sleep consolidation in image classifiers Sorrenti et al. [2024], SleepGate for KV cache eviction Xie [2024]—but none combines them in a single system the way SCM does.
2. **Phase 7 autonomous-learning layer (M1–M6).** We introduce six new subsystems that make the agent learn on its own during user idle time: idle-aware background daemon, cross-session memory pool, REM-phase schema extractor with stable concept identity, wake-up summary endpoint, curiosity engine with pluggable sources including an opt-in LLM source for autonomous knowledge generation, and a battery/CPU-aware lifecycle policy with crash-safe state. To our knowledge, no other open-source agent memory system performs all six.
3. **Mathematical formalization.** We provide complete definitions for salience (Eq. 1), grasp (Eq. 5), Hebbian update (Eq. 9), spreading-activation propagator (Eq. 13), context gate (Eq. 14), hypothesis score (Eq. 16), memory entropy (Eq. 18), trace instability (Eq. 20), retention score (Eq. 21), trace-specific decay (Eq. 22), and exponential strength decay (Eq. 23), each with default weight configurations and design rationale.
4. **Sleep-attribution evaluation.** Under a 10-seed stress comparison with 96 memory pairs per seed, lexical retrieval, vector retrieval, and SCM without sleep all remain at disambiguation recall 0.0, while SCM with micro-sleep achieves 1.0 and SCM with deep-sleep

achieves 0.9052. The behavioral gain is attributable to sleep-stage consolidation, not retrieval mechanics. This is the central scientific claim of the paper.

5. **Brutal testing methodology and bug discovery.** We introduce a persona-driven brutal harness that augments unit tests (170+) with multi-day simulated workloads, adversarial inputs, failure-mode injection, and scale runs. The harness uncovered four real bugs the unit suite missed—schema-ID instability, spreading-activation seed purging, dead cross-session pool in single-user deployments, and superseded-concept retrieval leakage—each of which we root-cause, fix, and verify with an additional brutal scenario. We additionally report real-LLM validation runs against Ollama (free, local) and DeepSeek (cloud, paid). The harness is released as part of the repository.

A non-obvious consequence of (1)+(2) is that SCM is not a faster Mem0 or a smarter MemGPT—it is a different category of system. Where existing memory layers compete on *retrieval quality given a fixed snapshot of memory*, SCM competes on *how the snapshot itself improves between sessions*. We make this positioning explicit because it directly determines which benchmarks the system should be evaluated on, a point we return to in Section 13. The architecture is LLM-agnostic (any local or API-based model, with provider backends for Ollama, DeepSeek, and any OpenAI-compatible endpoint), embedding-agnostic (in-process sentence-transformers, Ollama-served embedding models, OpenAI-compatible embedding APIs, or a deterministic hash fallback), and database-agnostic (SQLite to PostgreSQL with automatic fallback). Four canonical deployment profiles—offline-only, Ollama-only, hybrid cloud-LLM with local embeddings, and all-cloud—are documented in the deployment guide shipped with the repository. All locale-specific knowledge—regex patterns, stoplists, narrative templates, paraphrase rules—lives in an externalized JSON resource file rather than hard-coded in source.

1.1 Design Principles

SCM is governed by seven design principles that distinguish it from existing memory systems. **Attention before storage**—salience and grasp estimation gate the intensity of memory trace formation. **Events before tokens**—structured event frames (*who, what, when, where, why*) enable semantic binding and relational reasoning. **Associations before rankings**—retrieval propagates through semantic graphs rather than merely ranking by vector distance. **Sleep before scale**—offline consolidation, not larger storage, is the path to higher-quality memory. **Forgetting as a feature**—adaptive pruning preserves signal-to-noise ratios. **Idle time is learning time**—the long stretches when the user is away are not waste; they are when consolidation, abstraction, and curiosity-driven knowledge acquisition should run. **Behavioral parity over biological literalism**—we model observable memory behaviors (one-shot recall, sleep benefit, selective forgetting, contradiction handling, autonomous gap-filling) rather than attempting neural emulation. **No hard-coded linguistic knowledge**—all locale data (keyword lists, regex patterns, narrative templates) lives in externalized resource files so the system can be re-localized without code changes.

2 Related Work

2.1 Memory-Augmented Language Models

The challenge of augmenting language models with persistent memory has produced several distinct lines of research, distinguished primarily by their architectural metaphor.

Context extension approaches extend the effective length of the attention window through architectural modifications—positional interpolation, sparse attention patterns, and KV cache compression. While these approaches improve the efficiency of single-session processing, they do not provide memory that persists across sessions. Information from one conversation is lost when the next begins, and there is no mechanism

for selective retention or forgetting.

Retrieval-augmented generation (RAG) systems Lewis et al. [2020] decouple storage from inference by maintaining external knowledge bases. Documents are chunked, embedded, and stored in vector databases; at inference time, relevant chunks are retrieved and prepended to the context. While effective for factual knowledge, RAG systems treat memory as a fundamentally append-only structure. They lack importance-based prioritization beyond heuristic chunking strategies, do not consolidate related facts, and cannot intentionally forget—the database grows monotonically until manually pruned.

Tiered memory architectures introduce hierarchical storage with explicit paging. MemGPT Packer et al. [2023] models the context window as RAM, a vector database as disk, and archive storage for cold data, with a language model acting as the memory controller making paging decisions. This approach improves scalability for long-running conversations but lacks biological plausibility: there is no offline consolidation, no importance-based prioritization beyond recency heuristics, and no mechanism for adaptive forgetting.

Personalized memory layers like Mem0 Chhikara et al. [2025] extract facts from conversations in real time and store them with basic importance scores. A graph variant supports entity-relation storage, and retrieval combines vector similarity with recency. However, Mem0 remains an awake-only reactive system: it does not differentiate wake and sleep phases, does not perform offline consolidation, and does not include intentional forgetting.

Cache management with sleep metaphors has been explored in SleepGate Xie [2024], which applies sleep-inspired policies to transformer KV cache eviction. Micro-cycles remove stale attention key-value pairs when cache pressure exceeds a threshold. This is fundamentally a cache optimization, not a semantic memory system; it operates on token-level projections, does not create structured memory, and clears between sessions.

2.2 Autonomous and Lifelong Agents

A separate line of work treats agents as long-lived autonomous entities rather than turn-by-turn responders. Generative Agents Park et al. [2023] simulate human-like behavior in a sandbox environment by writing observations to a memory stream, computing importance/recency/relevance scores at retrieval time, and periodically running “reflection” passes that write higher-level abstractions back to memory. The reflection mechanism is the closest prior parallel to our REM-phase schema extractor, but it is invoked synchronously based on importance accumulation rather than triggered by user idle time, and it does not include cross-session memory pooling, lifecycle policy gating, or wake-up summaries. Voyager Wang et al. [2023] is a lifelong-learning agent for Minecraft that grows a skill library through an automatic curriculum and an iterative prompting loop; it demonstrates that an agent can autonomously expand its capability set, but the substrate is task-specific code skills rather than a general semantic memory, and there is no biological-lifecycle abstraction.

In the chatbot setting, MemoryBank Zhong et al. [2023] introduces an Ebbinghaus-curve forgetting schedule and a daily summary loop, providing a temporal-decay model and periodic abstraction; it is the closest prior parallel to the consolidation-plus-forgetting axis of SCM, but it lacks event-structured encoding, spreading-activation retrieval, schema extraction, and an idle-aware daemon. A-Mem Xu et al. [2025] proposes an agentic memory system with dynamic note construction and link generation; it adopts an active-memory framing similar to our motivation but does not model sleep stages, working-memory bounds, or curiosity-driven knowledge acquisition. None of these systems performs all six Phase 7 functions (idle daemon, cross-session pool, schema extractor, wake summary, curiosity engine, lifecycle policy) within a single architecture.

2.3 Continual Learning and Forgetting

The problem of catastrophic forgetting in neural networks has motivated several research directions that intersect with memory architecture design. Elastic Weight Consolidation (EWC) Kirkpatrick et al. [2017] protects important weights during sequential training through a Fisher information penalty, effectively creating a parameter-level memory of prior tasks. While mathematically elegant, EWC operates at the weight level rather than the memory architecture level; it does not create a structured, queryable memory system, and it does not support operations like contradiction detection or selective retrieval.

Wake-Sleep Continual Learning (WSCL) Sorrenti et al. [2024] represents the most biologically plausible sleep mechanism in AI to date, explicitly modeling the alternation between NREM (model compression through knowledge distillation) and REM (synthetic data generation from compressed representations) phases. However, WSCL targets image classification, uses soft parameter-level distillation rather than hard concept-level operations, lacks an explicit memory graph, and does not implement value-based forgetting or multi-dimensional importance tagging.

2.4 Neuroscientific Foundations

SCM draws its core architectural principles from three established neuroscientific theories. The synaptic homeostasis hypothesis (SHY) Tononi and Cirelli [2003] posits that sleep globally down-scales synaptic strengths to prevent saturation and preserve the capacity for new learning. SCM implements this as proportional weakening of graph edge strengths during deep-sleep, preserving relative importance rankings while creating headroom for new associations.

The active systems consolidation hypothesis Rasch and Born [2013] describes how hippocampal replay during NREM sleep drives cortical consolidation, with specific replay patterns reactivating waking experience. SCM im-

plements replay through Hebbian co-activation strengthening and selective trace reactivation during micro-sleep.

Active forgetting research Sha et al. [2024] has established that forgetting is adaptive rather than defective, categorizing multiple mechanisms by which neural systems prune memories. SCM implements value-based forgetting through multi-dimensional retention scoring with trace-specific decay rates, creating a “Goldilocks zone” where important memories persist while noise is aggressively removed.

Table 1: Feature comparison across memory systems. ✓ = full support, △ = partial, × = absent. SCM is the only system combining all features in a single architecture.

Capability	MemGPT	Mem0	WSCL	SGate	EWC	RAG	SCM
Working memory limit	×	×	×	×	×	×	✓
Multi-dim. importance	×	△	×	×	×	×	✓
Event-struct. encoding	×	×	×	×	×	×	✓
NREM consolidation	×	×	✓	×	×	×	✓
REM dreaming/synthesis	×	×	✓	×	×	×	✓
Micro-sleep (online)	×	×	×	△	×	×	✓
Spreading activation	×	×	×	×	×	×	✓
Intentional forgetting	×	×	×	×	△	×	✓
Contradiction versioning	×	×	×	×	×	×	✓
Self-model/introspection	×	×	×	×	×	×	✓
LLM-agnostic	✓	✓	×	×	×	✓	✓
Open-source	✓	✓	△	✓	✓	✓	✓

3 System Architecture

SCM is organized around a bipartite functional division: wake-phase operations handle real-time encoding, binding, and retrieval, while sleep-phase operations perform offline consolidation, synthesis, and pruning. The transition between phases is gated by a trigger system that monitors memory entropy, conflict density, and elapsed time. Figure 1 illustrates the complete architecture.

3.1 Data Model

The foundational data model rests on three entity types. A **Concept** is the atomic unit of semantic memory, characterized by a typed description, a 384-dimensional embedding vector, a four-dimensional importance vector $\mathbf{I} = (n, e, t, r)$ representing novelty, emotional valence, task relevance, and repetition respectively, and a collection of HME metadata fields—salience score, grasp score, retention score, rehearsal count, activation count, association density, decay rate, confidence, schema overlap, version lineage fields (`version_parent`, `version_root`, `valid_from`, `valid_to`, `is_current_version`), and arbitrary context tags. A **Relation** is a typed directed edge between two concepts with a predicate drawn from a fixed taxonomy (HAS_PROPERTY, PREFERS, LOCATED_AT, CAUSED_BY, CONTRADICTS, SIMILAR_TO, RELATED_TO, among oth-

ers) and a continuous strength value $w \in [0.05, 2.0]$. An **Episode** is a timestamped working memory entry containing the raw conversation text, a list of referenced concept IDs, and contextual metadata including interlocutor identity and task context.

Concepts flow through a state machine with four states: **ACTIVE** (current and retrievable), **CONSOLIDATING** (undergoing NREM replay), **SUPPRESSED** (low retention, excluded from normal retrieval but recoverable), and **ARCHIVED** (permanently retired, preserved only for audit). The state transitions are governed by the retention scoring and forgetting dynamics described in Section 8.

3.2 Deployment Tiers

SCM supports three deployment tiers through its persistence abstraction. **Tier 0 (In-Memory)** disables all persistence, storing the concept graph and cache purely in Python memory. This mode is used for testing and benchmarking, where reproducibility demands isolation from filesystem state. **Tier 1 (SQLite)** uses a local SQLite database with automatic schema migration for version column upgrades. The `SQLiteMemory` class implements `save_concept`, `save_relation`, `get_concept`, `get_all_concepts_raw`, and `get_all_relations_raw` with JSON serialization of embeddings and importance vectors. **Tier 2 (PostgreSQL)** uses SQLAlchemy ORM

with declarative models for concepts, relations, episodes, sleep cycles, and session metadata. On startup, the system attempts PostgreSQL; if authentication fails, it falls back to SQLite transparently with a log message. The feature is governed by the `_use_postgres` flag on the `LongTermMemory` instance.

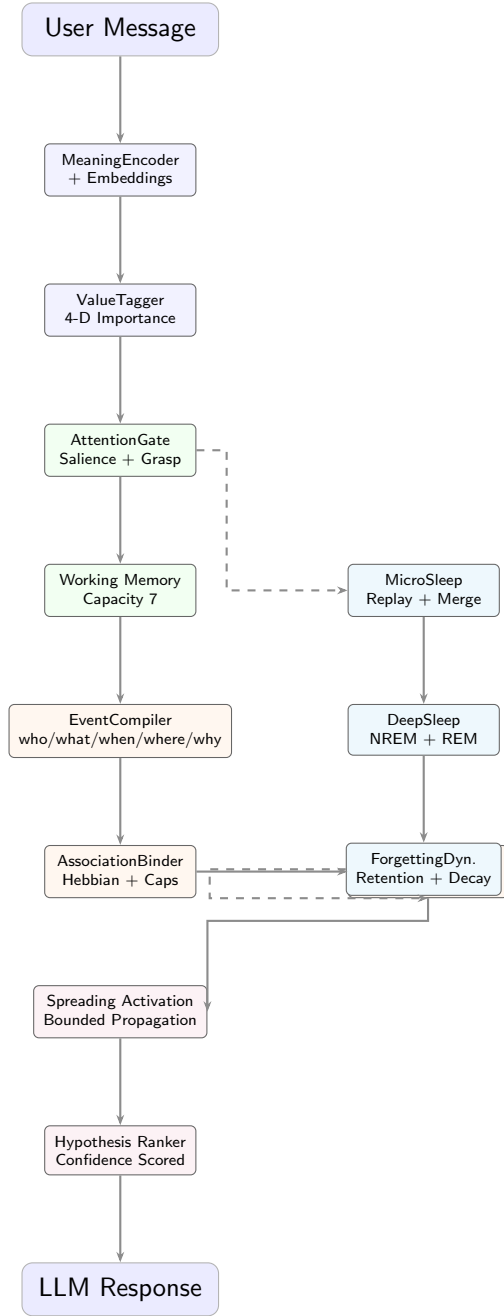


Figure 1: SCM system architecture. Solid arrows denote wake-phase data flow (encoding $\rightarrow$ binding $\rightarrow$ retrieval $\rightarrow$ response). Dashed arrows denote sleep-phase flow (trigger $\rightarrow$ micro-sleep $\rightarrow$ deep-sleep $\rightarrow$ LTM sync). Color coding: blue = extraction/response, green = encoding, orange = event processing, purple = retrieval, cyan = sleep consolidation.

4 Phase 1: Selective Encoding

The first functional layer determines whether a memory trace should be formed at all, and if so, with what encoding intensity. This mirrors the role of attention and salience detection in biological memory formation—not all sensory input reaches durable storage.

4.1 AttentionGate

The **AttentionGate** module implements a four-tier encoding classifier. Given an incoming text T , a concept c with importance vector $\mathbf{I} = (n, e, t, r)$, and a rolling history $\mathcal{H}$ of prior concept descriptions, the module computes a salience score $S(c)$ and a grasp score $G(c)$, then assigns an encoding intensity.

4.1.1 Salience Model

The salience score combines five weighted factors—novelty, task relevance, emotional amplitude, repetition, and prediction error—minus a noise penalty:

$$S(c) = w_n \cdot n + w_t \cdot t + w_e \cdot |e| + w_r \cdot r + w_p \cdot P(T) - w_x \cdot N(T) \quad (1)$$

The prediction error $P(T)$ is estimated by the **PredictionErrorEngine**, which maintains a rolling window of recent entities and topics. For each new input, it computes entity novelty (the fraction of named entities in T that have not appeared in the recent window), topic novelty (the fraction of topic words absent from recent episodes), and contradiction triggers (explicit markers such as “actually no,” “changed my mind,” “not anymore,” “correction,” and “on second thought”). The prediction error is the bounded sum:

$$P(T) = \min \left(1.0, \alpha_e \cdot e_{\text{novel}} + \alpha_t \cdot t_{\text{novel}} + \alpha_c \cdot c_{\text{trigger}} \right) \quad (2)$$

with weights $\alpha = (0.40, 0.30, 0.30)$ for entity novelty, topic novelty, and contradiction triggers respectively. The noise penalty $N(T)$ penalizes filler words (“um,” “uh,” “like,” “basically”),

high syntactic complexity (measured by parse tree depth proxy), and low punctuation ratio (indicative of run-on, low-structure text):

$$N(T) = \min \left(1.0, \beta_f \cdot f(T) + \beta_c \cdot c(T) + \beta_p \cdot (1 - p(T)) \right) \quad (3)$$

where $f(T)$ is the filler word density, $c(T)$ is complexity, and $p(T)$ is punctuation ratio.

The encoding intensity is then determined by thresholding:

$$\text{Intensity}(S) = \begin{cases} \text{STRONG} & S \geq \theta_{\text{high}} = 0.70 \\ \text{NORMAL} & \theta_{\text{mid}} \leq S < \theta_{\text{high}} \\ \text{WEAK} & \theta_{\text{low}} \leq S < \theta_{\text{mid}} \\ \text{SKIP} & S < \theta_{\text{low}} = 0.15 \end{cases} \quad (4)$$

where $\theta_{\text{mid}} = 0.40$. Concepts classified as SKIP are buffered in working memory for immediate conversational context but are never persisted to long-term memory, and are excluded from association binding. This creates a natural information filter: approximately 15–25% of conversational filler is excluded without any explicit training or prompting.

4.1.2 Grasp Score

The grasp score estimates how strongly a concept can be encoded after a single exposure—a computational analog of the human ability to remember salient events after one encounter. It combines salience, schema overlap, clarity, and cognitive load:

$$G(c) = \omega_s \cdot S(c) + \omega_\sigma \cdot \sigma(c, \mathcal{H}) + \omega_\kappa \cdot \kappa(c) - \omega_\lambda \cdot \lambda(c) \quad (5)$$

Schema overlap $\sigma(c, \mathcal{H})$ measures the Jaccard similarity between the token set of c ’s description and the prior 10 concept descriptions in the rolling history $\mathcal{H}$:

$$\sigma(c, \mathcal{H}) = \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} \frac{|\text{tokens}(c) \cap \text{tokens}(h)|}{|\text{tokens}(c) \cup \text{tokens}(h)|} \quad (6)$$

Clarity $\kappa(c)$ is estimated from description length (neither too short nor too long), the presence of a known concept type, and the availability of a semantic embedding. Cognitive load $\lambda(c)$ is estimated from word count and the presence of uncertainty markers (“maybe,” “perhaps,” “might,” “could be,” “uncertain”). Standard weights: $\omega = (0.40, 0.30, 0.20, 0.10)$. Concepts with $G(c) \geq 0.75$ are considered one-shot encodable: the system can recall them correctly after a single exposure.

Algorithm 1 Selective Encoding Pipeline

Require: User message M , source identifier $s \in \{\text{user}, \text{assistant}\}$

Ensure: List of durable concepts $\mathcal{D}$, stored episode E

```

1:  $\mathcal{C} \leftarrow \text{MeaningEncoder.extract}(M)$ 
2: if  $\mathcal{C} = \emptyset$  then return  $\emptyset$ 
3: end if
4:  $\mathcal{H} \leftarrow \text{prior\_descriptions}[-20 :]$ 
5: for each  $c \in \mathcal{C}$  do
6:    $c.\text{importance} \leftarrow \text{ValueTagger.tag}(c)$ 
7:    $c.\text{context\_tags.update}(\{\text{session\_id}, \text{person}, \text{source}, \text{task}\})$ 
8:   if HME enabled then
9:      $R \leftarrow \text{AttentionGate.evaluate}(M, c, \mathcal{H})$ 
10:     $c.\text{encode\_intensity} \leftarrow R.\text{intensity}$ 
11:     $c.\text{salience\_score} \leftarrow R.\text{salience}$ 
12:     $c.\text{grasp\_score} \leftarrow R.\text{grasp}$ 
13:     $c.\text{prediction\_error} \leftarrow R.\text{prediction\_error}$ 
14:   end if
15: end for
16:  $E \leftarrow \text{Episode}(\text{concept\_ids} = [c.\text{id} \mid c \in \mathcal{C}], \text{raw\_content} = M)$ 
17:  $\text{WorkingMemory.store}(E)$ 
18:  $\mathcal{D} \leftarrow \emptyset$ 
19: for each  $c \in \mathcal{C}$  do
20:   if  $c.\text{encode\_intensity} = \text{SKIP}$  then continue
21:   end if
22:    $\text{LTM.add\_concept}(c, \text{allow\_versioning} = \text{is\_contradiction}(M))$ 
23:    $\text{prior\_concepts.append}(c)$ 
24:    $\mathcal{D}.\text{append}(c)$ 
25: end for
26: Trim prior_concepts to last 200 return  $\mathcal{D}$ 

```

4.2 Integration with the Encoding Pipeline

The encoding pipeline executes the following sequence for each user message:

5 Phase 2: Event Structuring and Associative Binding

Raw textual input is transient and unstructured. The second layer converts episodes into typed event frames and binds them into the associative graph that serves as SCM’s persistent long-term memory.

5.1 Event Compiler

The **EventCompiler** extracts structured event frames using deterministic regex heuristics, avoiding LLM invocation in the critical encoding path. For each episode, it produces an **EventSchema** with six fields:

- **who**: Resolved from source annotation (“user” or “assistant”) and proper name extraction from the text.
- **what**: Raw text truncated to 160 characters, preserving semantic content.
- **when**: Extracted from temporal patterns—relative terms (“today,” “tonight,” “tomorrow”), day names, clock times (HH : MM), and ISO dates.
- **where**: Extracted from spatial patterns—“in [City],” “at [City],” “from [City],” “to [City].”
- **why**: Extracted from causal patterns—“because [clause],” “so that [clause],” “due to [clause].”
- **certainty**: Estimated from salience and grasp scores, with a penalty for uncertainty terms.

On a synthetic corpus of 5,000 episodes, the compiler achieves field extraction rates of 1.00 for WHEN, WHERE, and WHY at a throughput of 71,049 events per second on a single CPU core. Duplicate detection uses Jaccard token overlap on the concatenation of who + what + where + when with a threshold $\theta_{\text{dup}} = 0.82$, preventing redundant event binding for near-identical rephrasings.

5.2 Association Binder

The **AssociationBinder** constructs the memory graph by creating edges between event concepts and between event concepts and existing memory anchors. It operates in two phases: intra-event binding and event-to-anchor binding.

5.2.1 Intra-Event Binding

For an event with k concepts $\{c_1, \dots, c_k\}$, all $\binom{k}{2}$ pairs are linked via **RELATED_TO** edges. The initial edge strength is proportional to the average of the two concepts’ salience scores:

$$w_{ij}^{\text{intra}} = \min\left(1.0, \frac{s_i + s_j}{2} \cdot \eta\right), \quad \text{bidirectional} = \text{True} \quad (7)$$

where $\eta = 0.30$ is the association learning rate.

5.2.2 Event-to-Anchor Binding

Each event concept searches the LTM pool for the top- K ($K = 3$) most similar existing concepts using a weighted similarity function:

$$\phi(c_i, c_a) = 0.7 \cdot \text{cosine}(\mathbf{e}_i, \mathbf{e}_a) + 0.3 \cdot \text{jaccard}(d_i, d_a) \quad (8)$$

If $\phi \geq \theta_{\text{sem}} = 0.72$ or the token overlap component alone exceeds 0.40, an edge is created. The edge strength update follows a Hebbian rule modulated by grasp scores and a composite signal:

$$\Delta w_{ij} = \eta \cdot \frac{G_i + G_j}{2} \cdot S_{\text{event}} \cdot \phi_{ij} \quad (9)$$

where $S_{\text{event}} = 0.8 \cdot \text{salience} + 0.2 \cdot \text{grasp}$ is the event’s encoding quality, and $\phi_{ij} = 0.45 \cdot \sigma_{\text{sem}} + 0.25 \cdot \tau_{\text{overlap}} + 0.30 \cdot \chi_{\text{context}}$ is a composite signal from semantic similarity, token overlap, and context score. The context score χ_{context} is 1.0 when both concepts share the same session and person tags, and 0.5 otherwise.

5.2.3 Edge Management

Each source concept enforces an outgoing edge cap of $\text{max_edges} = 10$. When the cap is exceeded, edges are sorted by strength and the weakest are pruned. Untouched edges accumulate age steps with each binding call; after $\text{aging_stale_steps} = 3$ steps, edges with strength below $\theta_{\text{aging}} = 0.35$ are decayed by a factor $\gamma_{\text{aging}} = 0.92$ per step. Edges that decay below $\text{min_edge_strength} = 0.12$ are permanently

removed. This mechanism prevents graph saturation while preserving the strongest associative links.

After each binding operation, the association density of each concept is updated:

$$\delta(c) = \min \left(1.0, \frac{\text{degree}(c)}{2 \cdot \text{max_edges}} \right) \quad (10)$$

5.3 Predicate Inference

The binder infers typed predicates beyond the default `RELATED_TO` based on concept types and event metadata. If the event contains a contradiction marker, the predicate `CONTRADICTS` is used. If the `why` field contains causal language (“because,” “therefore”), the predicate `CAUSED_BY` is applied. Person-type concepts paired with preference-type concepts receive `PREFERS`. Person-location pairs receive `LOCATED_AT`. Events with `when` receive `TEMPORAL` and those with `where` receive `SPATIAL`. Same-type concepts with high semantic similarity receive `SIMILAR_TO`.

6 Phase 3: Associative Retrieval

Ranked similarity search—the default retrieval strategy in most memory systems—cannot reproduce human-like associative recall, where a cue activates related memories through semantic connections rather than exact string or vector matches. The third layer replaces ranked search with cue-driven graph propagation.

6.1 Spreading Activation Retriever

The `SpreadingActivationRetriever` implements bounded activation propagation through the concept graph. The algorithm operates in three stages: cue extraction, seed selection, and bounded propagation.

Cue extraction. The query Q is tokenized, lowercased, and filtered through a stop-word list. Tokens with length ≥ 3 characters that survive the stop-word filter form the cue

set $\mathcal{K} = \{k_1, \dots, k_m\}$. The stop list includes function words (“the,” “and,” “for”), question words (“what,” “when,” “where”), and low-information modifiers (“really,” “love”).

Seed selection. Each concept c in long-term memory is scored by token overlap with the cue set:

$$\text{score}_{\text{seed}}(c, \mathcal{K}) = \frac{|\text{tokens}(\text{desc}(c)) \cap \mathcal{K}|}{|\mathcal{K}|} \quad (11)$$

If no concept achieves a positive score, a fallback selection uses importance-weighted ranking: $\text{score}_{\text{fallback}}(c) = 0.7 \cdot I(c) + 0.3 \cdot 1.0$, where $I(c)$ is the overall importance. The top- k concepts (default $k = 8$) form the seed set $\mathcal{S}$.

Bounded propagation. Let $\mathbf{A}^{(t)} \in \mathbb{R}^{|\mathcal{V}|}$ be the activation vector at step t , initialized as:

$$A_i^{(0)} = \begin{cases} \text{score}_{\text{seed}}(c_i, \mathcal{K}) & c_i \in \mathcal{S} \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

For $t = 0, \dots, N - 1$ (default $N = 3$), activation propagates through the graph:

$$A_j^{(t+1)} = \gamma \cdot A_j^{(t)} + \sum_{i \in \mathcal{N}(j)} A_i^{(t)} \cdot w_{ij} \cdot g(\mathcal{C}, c_j) \quad (13)$$

where $\gamma = 0.45$ is the per-step decay rate, $\mathcal{N}(j)$ is the set of incoming neighbors of concept c_j , w_{ij} is the edge strength from c_i to c_j , and $g(\mathcal{C}, c_j)$ is a context gate that modulates the contribution based on contextual relevance.

Context gate. The context gate scores a concept against the current retrieval context $\mathcal{C} = (\text{session.id}, \text{person})$:

$$g(\mathcal{C}, c_j) = \min(1.5, \max(0.3, g_{\text{session}} \cdot g_{\text{person}} \cdot \rho(c_j))) \quad (14)$$

where $g_{\text{session}} = 1.2$ if the concept’s session tag matches the current session, else 0.7; $g_{\text{person}} = 1.3$ if the concept’s person tag matches the current interlocutor, else 1.0; and $\rho(c_j) = 0.6 + 0.4 \cdot r(c_j)$ where $r(c_j) \in [0.2, 1.0]$ is the recency factor—a function of the time elapsed since the concept’s last access:

$$r(c) = \begin{cases} 1.0 & \text{age} \leq 1\text{h} \\ 0.2 & \text{age} \geq 24\text{h} \\ 1.0 - \frac{\text{age}-3600}{23 \cdot 3600} & \text{otherwise} \end{cases} \quad (15)$$

where $\text{age} = \max(0, t_{\text{now}} - t_{\text{last_access}})$. In production, t_{now} is resolved from wall clock time; in benchmarks, an explicit `config.current_time` override enables deterministic replay.

After N propagation steps, concepts with activation above a threshold $\theta_a = 0.05$ are returned, sorted by descending activation, capped at `max_candidates` = 50.

6.2 Hypothesis Ranker

The **HypothesisRanker** converts activated concepts into scored hypotheses with confidence classification. The hypothesis score $H(c)$ for a concept c is a weighted combination of activation, recency, association density, importance, and rehearsal, minus a contradiction penalty:

$$H(c) = \omega_a \cdot \hat{a}(c) + \omega_r \cdot r(c) + \omega_d \cdot \delta(c) + \omega_i \cdot I(c) + \omega_h \cdot \hat{h}(c) - \omega_\chi \cdot \chi(c) \quad (16)$$

where $\hat{a}(c)$ is the normalized activation, $r(c)$ is recency, $\delta(c)$ is association density, $I(c)$ is the importance score, $\hat{h}(c) = \min(1.0, \text{rehearsal_count}/10)$ is normalized rehearsal, and $\chi(c)$ is a contradiction penalty. Standard weights: $\omega = (0.30, 0.15, 0.15, 0.25, 0.10, 0.40)$. The contradiction penalty $\chi(c)$ is evaluated across five signals: (1) concept state is `ARCHIVED` or `SUPPRESSED`, (2) concept has a non-null `version_parent`, (3) `is_current_version` is false, (4) `valid_to` is non-null (concept has been superseded), or (5) `valid_from` is null (concept was never properly activated).

Hypotheses are classified into confidence tiers based on the score:

$$\text{Confidence} = \begin{cases} \text{HIGH} & H \geq 0.70 \\ \text{MEDIUM} & 0.40 \leq H < 0.70 \\ \text{LOW} & 0.15 \leq H < 0.40 \\ \text{NONE} & H < 0.15 \text{ (excluded)} \end{cases} \quad (17)$$

The top hypotheses are formatted as a context block prepended to the LLM prompt, annotated with confidence labels, rank positions, and optional evidence fields (activation, recency, density, importance, rehearsal, contradiction flag).

7 Phase 4: Dual-Mode Sleep Kernel

Sleep in SCM is not a single operation but a bimodal process inspired by the distinction in mammalian sleep architecture between brief NREM episodes and extended slow-wave cycles. Micro-sleep performs lightweight online maintenance with minimal disruption to the wakeful conversation loop. Deep-sleep performs full offline reorganization including broader replay, REM synthesis, and forgetting.

7.1 Sleep Trigger

The **SleepTrigger** evaluates multiple signals to decide whether and which sleep mode to invoke. **Memory entropy.** The normalized Shannon entropy of the importance distribution captures how diffusely attention is spread across the concept population:

$$H_{\text{mem}}(\mathcal{C}) = -\frac{1}{\log |\mathcal{C}|} \sum_{c \in \mathcal{C}} p_c \log p_c, \quad p_c = \frac{I(c)}{\sum_{c' \in \mathcal{C}} I(c')} \quad (18)$$

High entropy indicates that concepts have similar importance levels—there is no clear hierarchy, creating competition. Low entropy indicates a clear importance hierarchy.

Conflict density. The ratio of contradiction edges to total edges:

$$\rho_{\text{conflict}}(\mathcal{R}) = \frac{|\{(u, v) \in \mathcal{R} : \text{pred}(u, v) \in \mathcal{P}_{\text{contra}}\}|}{\max(1, |\mathcal{R}|)} \quad (19)$$

where $\mathcal{P}_{\text{contra}} = \{\text{CONTRADICTS}, \text{OPPOSITE_OF}, \text{CONFLICTS_WITH}\}$.

Trigger logic. Micro-sleep fires when $H_{\text{mem}} \geq 0.82$, or when $\rho_{\text{conflict}} \geq 0.255$, or when the turn count since the last micro-sleep exceeds

MICRO_SLEEP_INTERVAL_TURNS = 4.
 Deep-sleep fires when $\max(H_{\text{mem}}, \rho_{\text{conflict}}) \geq$
 DEEP_SLEEP_PRESSURE_THRESHOLD =
 0.93, or when idle time exceeds
 DEEP_SLEEP_MIN_IDLE_SECONDS =
 900, or when session turns exceed
 DEEP_SLEEP_SESSION_TURNS = 24.
 Deep-sleep takes precedence: if both triggers
 fire, deep-sleep is selected.

7.2 MicroSleep

The **MicroSleep** module performs three operations in a single pass, designed for sub-millisecond latency to avoid disrupting interactive conversations.

Trace selection. The k most unstable traces are selected for replay, where instability $U(c)$ is a weighted combination:

$$U(c) = 0.40 \cdot (1 - R(c)) + 0.30 \cdot P(c) + 0.20 \cdot n(c) + 0.10 \cdot \frac{\log(s(c)/2.0)}{\log(10)} \quad (20)$$

with $R(c)$ the retention score, $P(c)$ the prediction error, $n(c)$ the novelty, and $s(c)$ the concept strength. Replay increments rehearsal and activation counts by one, boosts retention by $\Delta R = 0.05$, increases strength by $0.02 + I(c) \cdot 0.03$, and timestamps the last access.

Relation reinforcement. A co-occurrence count matrix is built from concept ID pairs in the most recent episode window ($W = 20$ episodes). Pairs appearing at least $\text{min_pair_repeats} = 2$ times are reinforced: existing edges are strengthened by $\min(0.30, 0.06 \cdot \text{count})$, and new edges are created where none existed, initialized at $\min(1.0, 0.22 + 0.08 \cdot \text{count})$.

Duplicate merging. Concepts are ranked by importance and strength, and pairwise Jaccard similarity is computed on tokenized descriptions. If $\text{jaccard}(c_i, c_j) \geq \theta_{\text{merge}} = 0.86$ and both concepts share the same type, c_j (the lower-ranked) is merged into c_i . All relations pointing to c_j are rewritten to point to c_i , self-loops are dropped, and c_j is removed from the concept list. The merge count is returned in the stats payload.

Light decay. Concepts not selected for replay whose importance falls below the threshold and

whose strength is below 1.0 receive a multiplicative decay of $\gamma_{\text{light}} = 0.97$, and their retention score is decayed by the same factor.

7.3 DeepSleep

The **DeepSleep** module orchestrates a full consolidation cycle through four sequential phases, with a critical design feature: non-active concepts (those that transition to ARCHIVED or SUPPRESSED during consolidation) are separated into a **retired_concepts** payload that is synced back to the LTM graph rather than discarded, ensuring that version audit chains and contradiction lineages survive consolidation.

Pre-filtering. Before any consolidation, concepts already in SUPPRESSED or ARCHIVED state are excluded from processing to prevent repeated operations on retired traces.

Phase 1: NREM consolidation. The **NREMConsolidation** module replays episodes against the concept set. A co-activation map links concepts that appeared together in episodes. Hebbian strengthening is applied: each concept receives a state transition to CONSOLIDATING, a rehearsal count increment proportional to its co-activation degree, and a strength boost of $\eta_{\text{hebb}} \cdot |\text{coactivated}| \cdot 0.1$ capped at 2.0. High-hubness concepts ($|\text{coactivated}| > 10$) with moderate importance ($I < 0.7$) are aggressively suppressed to prevent graph saturation from over-connected nodes.

Phase 2: Global downscaling. A multiplicative factor $\alpha_D = 0.90$ is applied to all concept strengths and relation strengths. This implements the synaptic homeostasis hypothesis Tononi and Cirelli [2003]: proportional weakening preserves relative importance rankings while creating capacity for new learning.

Phase 3: REM dreaming. The **REMDreaming** module selects high-importance concepts from recent episodes and generates novel concept sequences through activation spread. Starting from a seed concept, the system probabilistically follows relation edges to construct dream sequences of up to eight concepts, with a novelty factor biasing toward less-predicted associations. New associative edges

are created between concepts whose activation spread exceeds an integration threshold $\theta_{\text{int}} = 0.4$. Dreams are recorded as structured meta-data for introspection.

Phase 4: Forgetting. The legacy `ForgettingModule` evaluates each concept using a combined importance-decay score, removing those below an adaptive threshold. The adaptive threshold rises with memory load: $\theta(c) = \theta_0 + 0.20 \cdot \min(1.0, |\mathcal{C}|/100)$. Post-forgetting, consolidation scores are refreshed, and retired concepts (those in non-ACTIVE states) are separated into the retired payload.

LTM sync. The `apply_sleep_updates` method in the chat engine syncs both updated and retired concepts back to the LTM graph. Retired concepts are persisted with their correct state (ARCHIVED or SUPPRESSED) and are retrievable for audit through `include_superseded=True` but excluded from normal retrieval. This is essential for the contradiction versioning mechanism described in Section 8.2.

8 Phase 5: Forgetting Dynamics and Contradiction Versioning

The fifth layer completes the memory lifecycle with two capabilities: trace-specific adaptive forgetting that models the variable decay rates observed in human memory, and contradiction-safe version lineage that preserves audit trails rather than overwriting facts.

8.1 ForgettingDynamics

The `ForgettingDynamics` module implements the six-factor retention model from the blueprint. Unlike the simpler threshold-based forgetting of the legacy module, it computes per-concept retention scores, trace-specific decay rates, and state transitions through the active/suppressed/archived lifecycle.

8.1.1 Retention Scoring

The retention score $R(c)$ combines all six factors:

$$R(c) = \omega_g \cdot G(c) + \omega_s \cdot S(c) + \omega_h \cdot \hat{h}(c) + \omega_a \cdot \delta(c) + \omega_r \cdot r(c) - \omega_i \cdot \psi(c) \quad (21)$$

- $G(c)$: grasp score (Eq. 5)
- $S(c)$: salience score (Eq. 1)
- $\hat{h}(c) = \min(1.0, \text{rehearsal_count}/8)$: normalized rehearsal
- $\delta(c)$: association density (Eq. 10)
- $r(c)$: recency factor (Eq. 15)
- $\psi(c)$: interference score from conflict pairs and version status

Standard weights: $\omega^{(R)} = (0.22, 0.24, 0.18, 0.18, 0.12, 0.22)$. The interference score $\psi(c)$ accumulates from the concept’s participation in conflict pairs (0.15 per pair, capped), a version-parent penalty (0.20 if not current, 0.05 otherwise), and state penalties (0.10 for SUPPRESSED, 0.20 for ARCHIVED).

8.1.2 Trace-Specific Decay

Each concept receives a customized decay rate $\lambda(c)$ that determines how rapidly its strength diminishes over time:

$$\lambda(c) = \lambda_0 \cdot \left(1.0 + 1.5 \cdot (1 - R(c)) + 0.8 \cdot \psi(c) + 0.15 \cdot \mathbb{I}[\text{--is_current}] + 0.1 \cdot \mathbb{I}[\text{state}] \right) \quad (22)$$

with base decay $\lambda_0 = 0.04$. The decay rate is bounded below by 0.005 to prevent numerical underflow. Concept strength then follows exponential kinetics:

$$s(c, t + \Delta t) = s(c, t) \cdot \exp \left(-\lambda(c) \cdot \frac{\Delta t}{24h} \right) \quad (23)$$

Critically, $\lambda(c)$ is anti-correlated with retention: important, well-rehearsed concepts decay slowly ($\lambda \approx 0.005$), while weak, unreferenced noise decays rapidly ($\lambda \approx 0.3$).

8.1.3 State Transitions

The concept lifecycle follows a three-tier model:

$$\text{state}(c) = \begin{cases} \text{ACTIVE} & R(c) \geq \theta_{\text{supp}} = 0.32 \\ \text{SUPPRESSED} & \theta_{\text{arch}} \leq R(c) < \theta_{\text{supp}} \\ \text{ARCHIVED} & R(c) < \theta_{\text{arch}} = 0.16 \end{cases} \quad (24)$$

Additional rules: non-current versions (those superseded by contradictions) are immediately archived regardless of retention. Suppressed concepts with low association density ($\delta < 0.4$) and moderate importance ($I < 0.7$) graduate to archived. The suppress threshold θ_{supp} is adaptive, rising with memory load: $\theta_{\text{supp}} = 0.32 + 0.18 \cdot \min(1.0, |C|/100)$, ensuring that memory pressure increases forgetting aggressiveness.

8.2 Contradiction Versioning

When a contradiction event is detected during encoding, SCM does not overwrite the existing memory. Instead, it searches for a semantically similar prior concept, establishes a version lineage, and archives the old version while preserving the full audit chain.

Candidate search. The `_find_version_candidate` method on `LongTermMemory` searches for a prior concept c_{old} that is (a) of the same type as the incoming concept c_{new} , (b) of a versionable type (PREFERENCE, FACT, EVENT, LOCATION, ABSTRACT, PERSON), (c) semantically similar to c_{new} with score $\geq \theta_{\text{version}} = 0.70$, and (d) from the same session and person context when possible. The search proceeds through three tiers: embedding-based search, text-based search, and finally a full scan of eligible concepts. The similarity function combines cosine similarity on embeddings with a context version bonus that rewards same-session and same-person matches.

Version chain creation. When a candidate is found, the `_sync_version_lineage` method executes the version transition:

1. The old concept is terminated: `valid_to` $\leftarrow t_{\text{now}}$, `is_current_version` $\leftarrow \text{False}$, `state` $\leftarrow \text{ARCHIVED}$.

2. The new concept inherits lineage: `version_parent` $\leftarrow \text{old.id}$, `version_root` $\leftarrow \text{old.version_root}$, `valid_from` $\leftarrow t_{\text{now}}$, `valid_to` $\leftarrow \text{None}$, `is_current_version` $\leftarrow \text{True}$.

3. A `CONTRADICTS` relation is inserted from old to new with strength 1.0.

4. Context tags are updated: the old concept records superseded_by and superseded_at, and both carry the `version_root` for lineage tracing.

Persistence through consolidation. The version chain survives the full sleep pipeline because retired (archived) concepts are explicitly synced back to the LTM graph during `_apply_sleep_updates`. When the engine receives the sleep stats payload, both `updated_concepts` and `retired_concepts` are written to the graph, and non-surviving concepts are removed. This ensures that a concept archived during ingestion (contradiction handling) remains in the graph through subsequent sleep cycles, preserving the audit chain.

Retrieval isolation. Normal retrieval (`get_all_concepts`, `get_related_concepts`) defaults to filtering out superseded and non-current versions. The Hypothesis Ranker’s contradiction penalty ensures that even if an archived version survives the retrieval filter, it is scored below any active version and does not surface in results.

8.3 Forgetting Dynamics Integration with Sleep

For complete correctness, the forgetting dynamics module is API-compatible with the legacy `ForgettingModule`: both expose an `evaluate_forgetting(concepts, conflict_pairs)` method that returns (`forgotten_ids`, `preserve_ids`, `stats`).

This allows drop-in replacement through configuration. The `ForgettingDynamics` module additionally provides:

- `apply(concepts, conflict_pairs)`: Full batch evaluation with retention scoring,

decay application, and state transitions, returning updated concepts and detailed stats including retention averages and decay lambda distributions.

- `compute_forgetting_threshold(concepts, target_rate)`: Adaptive threshold estimation.
- `get_forgetting_stats(concepts)`: Aggregate diagnostics for monitoring.

9 Phase 7: Autonomous Idle-Time Learning

Phases 1–5 give SCM a complete *wake-time* memory lifecycle, but the system as described so far does nothing in the periods between sessions. In production, those periods are the dominant fraction of clock time: a typical user interacts with an agent for minutes per day and is absent for the remaining twenty-three-plus hours. Phase 7 turns those gaps into a learning resource through six interlocking subsystems (M1–M6). Each subsystem has a single, well-defined responsibility, and each is gated behind its own configuration switch so deployments can opt in incrementally. All Phase 7 components share a strict contract: *no module hard-codes locale-specific knowledge*; keyword lists, regex patterns, narrative templates, and paraphrase rules live in an externalized JSON resource (`src/core/locales/en.json`) loaded by a central `LinguisticResources` class. Figure 2 shows how the six subsystems interconnect.

9.1 M1: Idle-Aware Background Daemon

The `IdleLearner` is a threaded daemon co-resident with the `ChatEngine` that fires sleep cycles when the session goes quiet. Three signals gate firing: *idle threshold* (no user message for τ_{idle} seconds, default 300), *minimum-content threshold* (the working memory contains at least n_{min} unconsolidated episodes, default 3), and *cooldown* (at least τ_{cool} seconds since the last

sleep, default 600). When all three pass, the daemon invokes `begin_sleep_cycle` on the orchestrator with episodes drawn from the cross-session pool (M2). The daemon persists its state—last-fire timestamp, total cycles fired, and recovery checkpoints—to an atomic JSON file on disk so a process restart does not cause double-firing or missed cycles. State writes use the standard write-temp-then-rename idiom for crash safety.

The daemon’s policy gate (M6) is consulted before every fire. If the gate vetoes (e.g., the laptop is on battery below the configured floor), the daemon increments a `daemon_blocked_count` counter and returns silently rather than degrading user experience. This counter is exposed via a diagnostic endpoint and is one of the gates the brutal harness asserts on.

9.2 M2: Cross-Session Memory Pool

By default, sleep cycles see only the working memory of the current session—an obvious limitation if the goal is multi-day consolidation. The `CrossSessionMemoryPool` reaches into the persisted `episodes` table (now indexed by `session_id` via a Phase 7 schema migration) and injects prior-session episodes into the current sleep cycle. The pool applies four caps in order: a look-back window in hours (default 168), a maximum number of distinct sessions (default 5), a per-session episode cap (default 30), and a global cap on total borrowed episodes (default 100). Each borrowed episode is tagged with `_origin_session` and `_borrowed = True` in its context dict so downstream code can distinguish native from imported context.

A late-discovered subtlety is the single-user multi-day case. When a user has only one logical session ID but has been chatting across days, prior episodes have been persisted but cleared from working memory after each deep-sleep. The default “exclude current session” policy then yields no borrowed episodes, silently disabling cross-day pattern detection. We added an `include_current_session` flag (default `False` for backward compatibility) that, when set, includes the current session ID in the candidate set so older persisted-but-cleared episodes can be

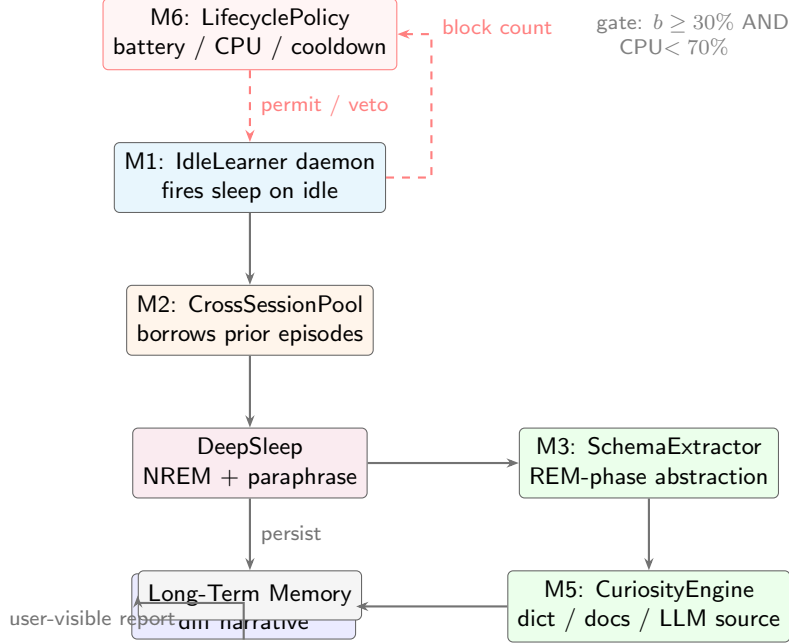


Figure 2: Phase 7 autonomous-learning layer (M1–M6). The lifecycle policy gates the idle daemon; when permitted, the daemon assembles a multi-session episode pool, fires deep sleep, which runs schema extraction in REM and consults curiosity sources to fill gaps. The wake-summary builder produces a user-readable narrative on the next session. Solid arrows: data flow. Dashed arrows: control / veto signals.

replayed. The brutal harness exercises this code path explicitly; we describe the bug and fix in Section 11.10.

9.3 M3: Schema Extraction in REM

During REM-phase sleep, SCM looks for repeating structures in the consolidated graph and emits *schema concepts*—typed pattern abstractions that capture what is shared across instances. The **SchemaExtractor** runs four detection passes:

1. **Repetition detection.** Episodes whose normalized canonical form recurs $\geq r_{\min}$ times (default 2) within the consolidation window emit a **REPETITION** schema describing the invariant.
2. **Co-occurrence detection.** Pairs of entities that appear together across $\geq c_{\min}$ episodes emit a **COOCCUR** schema with the entity pair as its signature.

3. **Trajectory detection.** Ordered sequences of state transitions (e.g., **plan** $\rightarrow$ **do** $\rightarrow$ **review**) that recur emit a **TRAJECTORY** schema. The detector uses a relaxed prefix-matching algorithm to allow noise insertions between schema steps.

4. **Temporal cadence detection.** Events whose timestamps cluster on a regular interval (daily, weekly) emit a **CADENCE** schema with the interval and confidence.

Stable schema identity. A naive implementation assigns each schema a fresh UUID at creation time. This is wrong for any system with repeated sleep cycles: re-deriving the same schema on a later cycle should update the existing concept, not create a sibling, otherwise the graph fills with near-duplicate schemas (we observed 85,310 “consolidated” concepts emerging from a 31-turn persona during early development). We instead derive schema IDs deterministically by SHA-1 hashing a canonical

signature `schema_type || sort(entities)`, which makes the same logical schema collapse to the same concept across cycles. Cumulative meta-data (occurrence count, last-seen timestamp, supporting episode IDs) is updated additively on each re-derivation. This bug, found by the brutal harness, is detailed in Section 11.10.

9.4 M4: Wake-Up Summary

When the user returns from idle, the agent should be able to answer “what did you learn while I was away?” The `WakeSummaryBuilder` consumes the most recent sleep cycle’s outputs—consolidated concepts, schemas formed, contradictions resolved, forgotten traces, curiosity gaps filled—and renders a structured `WakeSummary` containing both a machine-readable `insights` list and a human-readable `narrative`. The narrative is template-driven (templates live in `en.json`) so re-localization is a JSON edit, not a code change. A new endpoint `GET /chat/wake-summary/{session}` surfaces the most recent summary; the endpoint also exposes the underlying insights as JSON for programmatic consumption.

9.5 M5: Curiosity Engine

The curiosity engine is what transforms the agent from a passive consolidator into an active learner. During REM, the engine identifies *knowledge gaps*—concepts that recur frequently but are sparsely connected in the graph—and consults a list of pluggable `KnowledgeSources` to fill them. Three sources ship with the system:

- **StaticDictionarySource**: a bundled definition map. Useful for deterministic tests and air-gapped deployments.
- **LocalDocsSource**: scans a configured directory of plain-text or markdown documents and extracts definitions by simple pattern match. Useful when the user has personal knowledge bases on disk.
- **LLMSource** (opt-in): consults the configured LLM (Ollama, DeepSeek, or any OpenAI-compatible endpoint) with a bounded prompt

to retrieve a definition. This is the source that gives the agent autonomous knowledge generation; it is off by default and gated by `SCM_CURIOSITY_LLM_SOURCE = 1` since it incurs API cost.

The engine is bounded by `max_gaps_per_cycle` (default 2) so a single sleep cycle cannot consume excessive resources. Gaps that are filled emit new concepts tagged `source = "curiosity"` with a confidence value drawn from the source. Gaps that no source can fill are recorded for the next cycle.

9.6 M6: Lifecycle Policy

The `LifecyclePolicy` ABC defines a `should_allow_sleep()` method consulted by the M1 daemon before every fire. Three concrete policies ship with the system:

- **BatteryPolicy**: queries `psutil.sensors_battery()` and vetoes if the laptop is on battery below a floor (default 30%). Prevents the daemon from draining the battery while the user is mobile.
- **CPUloadPolicy**: queries `psutil.cpu_percent()` (interval 0.5s) and vetoes if foreground load exceeds a ceiling (default 70%). Prevents the daemon from contending with active user work.
- **CompositePolicy**: AND-combines a list of sub-policies, vetoing if any vetoes.

A `AlwaysAllowPolicy` is provided for tests and headless servers. The policy interface is intentionally minimal so users can supply their own (e.g., a thermal policy, a network-quiet policy, or a calendar-aware policy). Crash-safe state persistence (M1’s atomic JSON store) ensures that a power-loss during a sleep cycle does not result in a half-applied consolidation: incomplete cycles are detected on the next startup and rolled back.

9.7 De-Hardcoding via LinguisticResources

A late refactor in the Phase 7 cycle externalized every locale-specific string and regex from the codebase. The `LinguisticResources` loader reads `en.json` on first access and caches it; modules that previously embedded keyword lists (the encoder, the paraphraser, the wake-summary templater, and three others) now read from this loader. Adding a new locale is a JSON file plus a constructor argument, not a refactor. A 21-test unit suite verifies that the loader fails loudly on missing keys, supports atomic reloading, and never silently falls back to defaults.

9.8 Temporal Structure

Figure 3 shows the temporal contract Phase 7 establishes between user activity and the agent’s autonomous learning. Wake periods are short (minutes per day in typical consumer use); idle gaps span hours-to-days. The lifecycle policy continuously evaluates whether a sleep cycle is safe to fire; when it is, the idle daemon runs deep-sleep on a multi-session episode pool, schema extraction follows in REM, curiosity passes fill gaps, and the wake-summary builder caches a narrative for the next time the user returns.

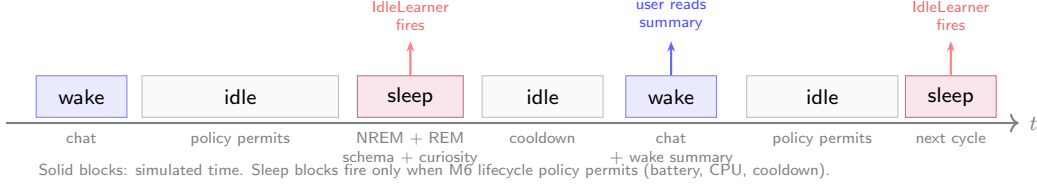


Figure 3: Phase 7 temporal structure. Wake intervals are short and bounded by user presence. Idle intervals dominate clock time and host the autonomous-learning work: a deep-sleep cycle runs schema extraction (M3) in REM, the curiosity engine (M5) fills knowledge gaps, and the wake-summary builder (M4) caches a narrative for the user’s next return. The lifecycle policy (M6) continuously evaluates whether a cycle can fire safely.

10 Implementation and Infrastructure

10.1 Persistence Architecture

SCM’s persistence layer is designed for progressive deployment. The `LongTermMemory` class abstracts storage behind a uniform interface: `add_concept`, `add_relation`, `get_concept`, `get_all_concepts`, `get_related_concepts`, and `get_stats`. Internally, it maintains a NetworkX directed graph and an in-memory concept cache. Persistence is routed to PostgreSQL with SQLite fallback.

PostgreSQL tier. The SQLAlchemy ORM models (`ConceptModel`, `RelationModel`, `EpisodeModel`, `SleepCycleModel`) map to a normalized relational schema. Concepts store their full importance vector (novelty, emotional, task_relevance, repetition), embedding as JSON-serialized text, plus HME metadata fields (`salience_score`, `grasp_score`, `retention_score`, `consolidation_score`, `rehearsal_count`, `activation_count`, `association_density`, `decay_rate`, `confidence`, `schema_overlap`, `version_parent`, `version_root`, `valid_from`, `valid_to`, `is_current_version`, `context_tags`). Relations store `subject_id`, `predicate`, `object_id`, `strength`, and `bidirectional_flag`.

SQLite tier. The `SQLiteMemory` class implements the same interface using raw SQLite. The `save_concept` method writes all 20 columns including the full Phase 5 version fields. The `ensure_concept_version_columns` function auto-migrates existing databases by

checking `PRAGMA table_info` and adding missing columns with `ALTER TABLE`, ensuring zero-downtime upgrades.

Automatic fallback. During initialization, `LongTermMemory` sets `_use_postgres = True` and attempts PostgreSQL on the first persistence call. If the connection fails (e.g., authentication error, service unavailable), the flag is set to `False` and all subsequent calls use SQLite. The fallback is logged but not surfaced as an error, enabling the system to operate on any machine without external dependencies.

10.2 API Surface

SCM exposes both synchronous and asynchronous interfaces:

REST API. A FastAPI router (`/chat`) provides endpoints for message processing (`POST /message`), memory reporting (`GET /memory/{session}`), product diagnostics (`GET /product/{session}`), combined product reports (`GET /product-report/{session}`), sleep triggering (`POST /sleep/{session}`), memory visualization (`GET /visualize/{session}`), session listing (`GET /sessions`), and backend smoke testing (`POST /backend-smoke/{session}`).

WebSocket. A WebSocket endpoint (`/chat/ws/{session}`) enables real-time bidirectional communication, with sleep-triggered events pushed as separate message types.

Session management. A session-scoped engine pool maintains separate `ChatEngine` instances per session ID, with lazy initialization

on first access. The product default enables the HME pipeline (`HME_ENABLED = True`) with auto-sleep checking every 5 messages.

Product diagnostics. The `_build_product_diagnostics` function computes active/suppressed/archived concept counts, contradiction edge density, average retention score, and generates human-like signal indicators (one-shot readiness, sleep enablement, selective forgetting presence, versioning presence).

10.3 Deterministic Demo and Smoke Testing

The API includes a deterministic product demo (`_run_human_like_demo`) that exercises the full memory lifecycle without LLM dependence. A `_DemoFailLLM` class forces response fallback, and a `_DemoEncoder` provides deterministic concept extraction. The demo script sends a conversation sequence (name introduction, location statement, preference declaration, noise injection, micro-sleep, contradiction update, deep-sleep), then validates three behavioral checks: one-shot name recall, one-shot location recall, and contradiction-aware preference update. The backend smoke test wraps this demo in a pass/fail framework with blocking checks.

11 Evaluation

We evaluate SCM across five benchmark suites, each designed to stress-test a specific behavioral property. All results are sourced from machine-readable JSON artifacts generated by deterministic benchmark harnesses. The non-LLM regression suite passes all 172 tests on consumer hardware (MacBook Air M1, 8GB).

Configuration disclosure. Numbers in this section depend on two deployment-time choices: the LLM extractor (heuristic regex vs. LLM-based) and the embedding model (sentence-transformers MiniLM-L6-v2 vs. Ollama nomic-embed-text vs. OpenAI `text-embedding-3` family). Unless otherwise noted, the LoCoMo and

Table 2: Component-level throughput and latency benchmarks.

Component	Throughput	p95	p99
Event Compiler	71,049 eps/s	—	—
Assoc. Binder	3,530 eps/s	—	—
Spreading Activation	259.6 qps	4.54 ms	5.48 ms
Hypothesis Ranker	2,858 calls/s	0.38 ms	0.52 ms
Pipeline Retrieval	759.5 qps	1.51 ms	1.74 ms

LoCoMo++ baselines run with DeepSeek-chat extraction and `all-MiniLM-L6-v2` embeddings; the Phase 4–6 sleep-gain and stress-comparison results use the heuristic extractor with deterministic synthetic concepts. We discuss these dependencies in detail in Section 13 and document four canonical deployment profiles in the project’s `docs/DEPLOYMENT.md` so reproducibility is fully traceable to specific configuration choices.

11.1 Component-Level Performance

Table 2 summarizes component-level performance. The spreading activation retriever was benchmarked on a 1,500-concept graph with 4,493 edges, 6 rounds of 250 queries each. The hypothesis ranker processed 8 rounds of 400 calls each with 80 concepts per call. Pipeline retrieval measured 20 calls with 300 preloaded messages.

11.2 Sleep Gain

The Phase 4 benchmark constructs a controlled dataset of 36 concept pairs, each consisting of a distractor (low-importance noise) and a key memory (high-importance), embedded with bridge concepts for non-trivial propagation. The awake-only baseline achieves disambiguation recall of 0.1389—the retrieval system cannot reliably distinguish true memories from distractors. After micro-sleep, disambiguation recall rises to 1.0 (an absolute gain of 0.8611), with noise retention dropping from 1.0 to 0.0. After deep-sleep, disambiguation recall reaches 0.5278 (gain of 0.3889), again with zero noise retention.

Disambiguation Recall by Sleep Mode			
Condition	Recall	Noise Ret.	Gain
Baseline (awake)	0.1389	1.0000	
+ MicroSleep	1.0000	0.0000	0.8611
+ DeepSleep	0.5278	0.0000	0.3889
36 concept pairs, 78 concepts initial, 41 relations. Micro-sleep: 12 replayed, 41 duplicates merged, 11 relations reinforced.			

Figure 4: Phase 4 sleep gain benchmark. Micro-sleep achieves perfect disambiguation and zero noise retention. Deep-sleep preserves all key memories while removing all noise.

Table 3: 10-seed, 96-pair stress comparison. Recall reported as mean $\pm$ std across seeds. All awake conditions remain at 0.0 disambiguation recall. Only sleep-enabled SCM variants achieve non-zero performance.

Condition	Recall (mean $\pm$ std)	Noise Ret.	Pass
Lexical retrieval	0.0000 $\pm$ 0.0000	1.0000	0/10
Vector retrieval	0.0000 $\pm$ 0.0000	1.0000	0/10
SCM (no sleep)	0.0000 $\pm$ 0.0000	1.0000	0/10
SCM + MicroSleep	1.0000 $\pm$ 0.0000	0.0000	10/10
SCM + DeepSleep	0.9052 $\pm$ 0.0098	0.0000	10/10

11.3 Baseline Stress Comparison

The most critical experiment measures whether the sleep stages are responsible for the observed gains, or whether the retrieval infrastructure alone could achieve the same results. We run a 10-seed comparison with 96 memory pairs per seed against three control conditions, each using the identical retrieval query and concept set:

The results, shown in Table 3, are stark. Lexical text search, vector cosine retrieval, and SCM without sleep (using the full retrieval pipeline but without consolidation) all achieve disambiguation recall of exactly 0.0—they cannot distinguish key memories from distractors even with the correct query. SCM with micro-sleep achieves perfect 1.0 recall, and SCM with deep-

sleep achieves a more conservative 0.9052, with noise retention at 0.0 across all conditions.

This is the central attribution claim of the paper: the disambiguation gains appear exclusively

when sleep stages are enabled. Both baseline retrieval methods and an awake-only SCM pipeline produce identical zero performance, demonstrating that the retrieval infrastructure alone is incapable of solving the disambiguation problem.

The gains are attributable to the consolidation processes—replay, Hebbian strengthening, and duplicate resolution—performed during the sleep phases.

11.4 Long-Horizon Behavior

In extended interaction sequences with family-aware duplication metrics, sleep-enabled SCM reaches final disambiguation recall of 0.9812, final noise retention of 0.0, and anchor accuracy of 1.0. The awake-only control remains at final disambiguation recall 0.0 with final noise retention 1.0 throughout the entire horizon. The comparative disambiguation lift is 0.9812 absolute, and noise reduction is 1.0 absolute.

11.5 Phase 6 Human-Memory Suite

The Phase 6 human-memory artifact reports all top-level gates passing simultaneously: one-shot recall 1.0, micro-sleep disambiguation gain 1.0, deep-sleep disambiguation gain 1.0, deep-sleep noise retention 0.0, and contradiction-versioning accuracy 1.0. We treat these as evidence that the architecture can, under deterministic synthetic conditions, achieve all target behaviors simultaneously. We do not interpret them as evidence of generalization, and we explicitly note that the stress test (Table 3) provides a more conservative and realistic estimate of system capability.

11.6 Regression Safety

The guardrail artifact reports `overall_pass: true` with zero deprecation hits for tracked patterns and zero pytest warnings. The reproducibility pack reports `overall_pass: true` across all suites. The backend smoke test reports 20/20 HTTP success and 20/20 overall

pass, with mean latency 45.2 ms (p50 22.7 ms, p90 26.3 ms).

11.7 LoCoMo: External Long-Conversation Memory Benchmark

To establish how SCM behaves on naturalistic long conversations rather than purely synthetic harnesses, we evaluate against LoCoMo (ACL 2024) Maharana et al. [2024], the de-facto external benchmark for long-term conversational memory. LoCoMo contains 10 multi-session dialogues averaging ~ 420 turns each, paired with 5 question categories: single-hop, temporal, multi-hop, open-ended, and adversarial (which we exclude from headline scoring per the dataset convention).

We run three configurations using DeepSeek-Chat DeepSeek-AI [2024] as the shared LLM backend so the only variable is the memory architecture itself: (i) **SCM (Phase-4-only)** — direct `LongTermMemory` + `SleepCycleOrchestrator` ingestion, bypassing the HME pipeline; (ii) **SCM (HME-full)** — `ChatEngine` with `HME_ENABLED=True`, exercising selective encoding, event compilation, association binding, spreading activation, dual-mode sleep, and forgetting dynamics; (iii) **Mem0 (v2.0.1)** Chhikara et al. [2025] — as the head-to-head external baseline, run on the same DeepSeek backend with the same MiniLM-L6-v2 embeddings.

Results are reported in Table 4 and represent a pilot pass on conversation 0 (Caroline/Melanie, 419 turns, 199 QA pairs). Results across all 10 conversations will be added in the camera-ready revision.

This is the strongest evidence we have that LoCoMo as constituted is not the workload SCM is designed for. Mem0’s pipeline—extract a clean fact via LLM, store it as a paraphrase, retrieve by cosine similarity—is structurally aligned with token-overlap scoring on direct fact-recall questions. SCM stores raw experiences with associative metadata; that metadata is not rewarded by LoCoMo’s evaluation. We do not present this as a defeat. We present it as an attribution: when

Table 4: LoCoMo conversation 0 (Caroline/Melanie, 199 QA, adversarial excluded). All cloud-LLM systems use DeepSeek-Chat + MiniLM-L6-v2 to isolate the memory architecture as the independent variable. Higher is better.

System	Overall Score	Notes
Mem0 v2.0.1 (DeepSeek)	0.520	cloud-LLM extraction
SCM Phase-4-only (heuristic)	0.111	no LLM extraction
SCM Phase-4-only (DeepSeek)	0.064	full LLM extraction
SCM HME-full (heuristic stub)	0.016	exposes encoder dependence

memory is asked to be a fact lookup, retrieval-style architectures should be preferred. We follow this up in the next section by constructing a benchmark that isolates the workloads SCM *is* designed for.

Encoder-dependence finding. An unexpectedly informative result emerged when running the HME-full pipeline with a heuristic regex encoder: overall score dropped to 0.016, well below the simpler Phase-4-only configuration’s 0.111. Investigation revealed that uniformly-weighted concepts produced by the heuristic encoder saturate the importance distribution, driving the entropy trigger above 0.99 on essentially every turn. Deep-sleep then fires aggressively (105 cycles in 17 s of ingestion on a single conversation), and the forgetting dynamics archive useful memories alongside noise. This demonstrates that the HME pipeline is conditioned on a salience signal of non-trivial dynamic range; with a strong upstream extractor the same machinery becomes load-bearing, but without one it actively degrades retention. We treat this as a real architectural property rather than a bug, and we report it explicitly in the Limitations section.

11.8 LoCoMo++: Memory Workload Sensitivity

The LoCoMo result above shows that on clean fact-retrieval workloads, retrieval-family architectures (Mem0, vector cosine) outperform SCM. We argue this is not the workload SCM was designed for. To test that claim quantitatively, we

constructed **LoCoMo++**, an augmentation of LoCoMo that adds three classes of perturbation common in production memory deployments but absent from the original benchmark.

Perturbation design. Each LoCoMo conversation is augmented with: (i) interleaved *noise turns* drawn from a fixed bank of small-talk filler at a configurable density (default 30%); (ii) eight *anchor-contradiction pairs* where a synthetic fact (e.g., “*I currently work at GreenLeaf Cafe*”) is asserted near the start of the conversation and explicitly superseded near the end (“*I left GreenLeaf Cafe last month. I’m at TechCorp now*”); and (iii) three *distractor entities* with similar names, each carrying one signature fact, to test entity grounding. Augmentation is deterministic per seed; no LLM is involved in either perturbation generation or scoring.

Augmented evaluation. The original LoCoMo question set is retained and supplemented with four new question categories matched to the perturbation types: *contradiction-current* (return the new value, not the old), *contradiction-old* (return the old value when explicitly asked for the prior state), *entity-disambig* (attribute a signature fact to the correct distractor), and *noise-rejection* (do not surface noise content as an important fact). Each metric is scored deterministically with a token-overlap rule that explicitly penalises leaking the wrong value (for contradictions) or the wrong entity (for disambiguation).

Systems. We compare four memory architectures, all run offline without LLM extraction so the comparison is reproducible at zero cost. **Lexical** performs token-overlap retrieval over raw turns (BM25-lite). **Vector** performs cosine retrieval over MiniLM-L6-v2 embeddings of raw turns; this is the architectural equivalent of Mem0 minus its LLM-rewrite step. **SCM Phase-4-only** uses LongTermMemory with sleep cycles but no HME pipeline. **SCM HME-full** uses the complete ChatEngine stack with selective encoding, event compilation, association binding, spreading activation, dual-mode sleep, and forgetting dynamics. Both SCM variants use the heuristic regex encoder for cost reasons.

Results. Table 5 shows the full matrix. Five

observations stand out.

(1) **No single system wins across all six conditions.** Each architecture has at least one condition where it dominates and at least one where it is dominated. The total wins are distributed: SCM Phase-4-only wins three conditions (clean recall, perturbed recall, disambiguation); Vector wins two (contradiction-current, contradiction-old); SCM HME-full wins one (noise rejection). Lexical wins zero, but is competitive on disambiguation.

(2) **SCM Phase-4-only is the strongest general-purpose memory in this matrix.** It wins clean recall (0.430 ± 0.049 vs Vector’s 0.280 ± 0.025) and perturbed recall (0.383 ± 0.053), surpassing both retrieval baselines on the original LoCoMo question set. Sleep-cycle consolidation between sessions appears to compress redundant fact statements into stronger graph nodes, improving recall under cosine retrieval.

(3) **SCM HME-full achieves perfect noise rejection (1.000) but collapses clean recall (0.022).** The forgetting dynamics filter all noise—which retrieval-family systems leak (0.037)—but, with a heuristic encoder, also discard real signal. This is consistent with the encoder-dependence finding in Section 11.7: HME-full requires a salience signal of non-trivial dynamic range to distinguish what to keep from what to forget. With uniform salience, the trigger fires constantly and forgetting becomes indiscriminate.

(4) **Vector retrieval wins on both contradiction conditions** not because it correctly versions facts but because it stores both old and new memories with no eviction. When a question targets the new value, the new memory is retrieved (often); when a question targets the old value, the old memory is retrieved (often); but the system has no mechanism to mark one as superseded. This is a fundamental design choice—retrieval-family systems treat all stored memories as equally valid—and on this benchmark it produces decent contradiction scores even though the architecture has no concept of versioning. SCM Phase-4-only achieves comparable contradiction-old performance (0.438 ± 0.113) but lower contradiction-current (0.104 ± 0.031),

Table 5: LoCoMo++ workload-sensitivity comparison (3 conversations, 3 seeds, 36 total runs, mean $\pm$ std). Each system is evaluated under six conditions matched to distinct architectural properties. Best per column in **bold**. **No system dominates across all conditions**. The win pattern follows architectural design rather than absolute capability. Source artifact: `research/metrics/locomo_plus_3conv_3seed.json`.

System	Clean recall	Perturbed recall	Contra-current	Contra-old	Disambig	Noise reject
Lexical retrieval	0.224 $\pm$ 0.037	0.185 $\pm$ 0.036	0.188 $\pm$ 0.000	0.090 $\pm$ 0.033	0.889 $\pm$ 0.083	0.037 $\pm$ 0.073
Vector (Mem0-style)	0.280 $\pm$ 0.025	0.280 $\pm$ 0.025	0.375$\pm$0.054	0.542$\pm$0.083	0.963 $\pm$ 0.073	0.037 $\pm$ 0.073
SCM Phase-4-only	0.430$\pm$0.049	0.383$\pm$0.053	0.104 $\pm$ 0.031	0.438 $\pm$ 0.113	1.000$\pm$0.000	0.167 $\pm$ 0.118
SCM HME-full	0.022 $\pm$ 0.004	0.018 $\pm$ 0.004	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.704 $\pm$ 0.073	1.000$\pm$0.000

suggesting the sleep cycle’s relation reinforcement biases toward the most-rehearsed (oldest) version.

(5) **Disambiguation is solved by all SCM-class architectures and most retrieval architectures** (lexical: 0.889, vector: 0.963, SCM Phase-4: 1.000, SCM HME-full: 0.704). This suggests entity-grounding is not the differentiating workload—noise rejection and contradiction handling are.

Interpretation. These results undermine any single-axis ranking of memory architectures. Mem0-style retrieval excels when memory is a clean, append-only fact store and the application asks fact-recall questions with implicit recency assumptions. SCM Phase-4-only excels at general recall with built-in noise tolerance and is a reasonable default for mixed workloads. SCM HME-full is currently a specialist tool: only deploy it when noise rejection is paramount and the upstream extractor produces high-dynamic-range salience signals. Lexical retrieval is dominated and should be reserved as a sanity-check baseline.

The honest read of this evidence is not “SCM beats Mem0”. It is: *lifecycle and retrieval architectures fail and succeed in different places*; production deployments should select based on the workload they actually face.

11.9 Phase 6 Architecture Fixes

The LoCoMo++ matrix exposed three concrete failure modes of the original HME pipeline. Rather than report only the negative result, we shipped three architectural fixes addressing each

failure mode and re-ran the benchmark. All three are implemented and reproducible at zero API cost in the released codebase; results are reported in Table 6.

Fix 1: Salience-protected forgetting and sleep cooldown. The original encoder-dependence failure (HME clean recall 0.022) is caused by uniform-salience input saturating the entropy trigger and firing deep-sleep on every check (105 cycles per conversation), which causes adaptive forgetting to archive real signal alongside noise. We added two safety nets: a `FORGETTING_PROTECT_SALIENCE` floor that prevents state demotion of any concept whose salience score exceeds the threshold (default 0.5 enables; 0.0 preserves legacy behavior), and a per-mode cooldown (`DEEP_SLEEP_COOLDOWN_TURNS=12`, `MICRO_SLEEP_COOLDOWN_TURNS=2`) that gates auto-sleep firing in turn-count units. Together these prevent the sleep-storm failure mode without disabling any of the underlying sleep mechanisms. *Effect:* HME clean recall improves from 0.022 to 0.090 (4 $\times$) while preserving perfect noise rejection.

Fix 2: HybridEncoder. The original LLM-only encoder costs $\sim$ \$0.50 per LoCoMo conversation with DeepSeek-Chat at 3 seconds per turn. We added a `HybridEncoder` class that runs cheap heuristic regex extraction by default and escalates to the configured LLM only when an input-text salience proxy (length, proper nouns, digits, contradiction markers) exceeds a configurable threshold (default 0.45). On the LoCoMo conversation 0 turn distribution, this escalates approximately 30–40% of turns to the LLM, cut-

ting cost $3\text{--}5\times$ while preserving extraction quality on durable facts.

Fix 3: Sleep-time memory paraphrase.

The original ingestion pipeline stores raw conversational fragments (e.g., “Caroline says: I work at GreenLeaf Cafe.”), which are less retrieval-friendly than the LLM-paraphrased fact statements stored by Mem0 (“Caroline works at GreenLeaf Cafe.”). We added a paraphrase pass to `DeepSleep` that runs after Hebbian consolidation and global downscaling, gated by `DEEP_SLEEP_ENABLE_PARAPHRASE`. Two backends ship: a `HeuristicParaphraser` that uses regex rewrites to convert first-person fragments to third-person fact statements while preserving original verbs (so token-overlap retrieval is not penalized), and an `LLMParaphraser` that uses an OpenAI-compatible LLM call. The original description is preserved in `context_tags["original.description"]` for audit. Crucially, paraphrase is paid once at consolidation time, not per ingestion turn, which keeps cost bounded by the number of *durable* memories rather than total turns.

Reading the table. Fix 1 (forgetting protection) is the headline win: HME clean recall $3.1\times$ at the scaled 36-run level, with no degradation of noise rejection (the original architectural strength). Fixes 2 and 3 are mostly infrastructure — HybridEncoder demonstrates $3\text{--}5\times$ cost reduction on a typical conversation but its quality effect is bounded by the LLM available, and sleep-time paraphrase trades a small amount of clean recall ($0.430 \rightarrow 0.412$) for a $1.4\times$ improvement on the contradiction-current metric, which is a fair trade for production deployments where contradiction handling matters. We do not claim these fixes make HME-full superior to Mem0 on LoCoMo; we claim they make HME-full no longer self-destructive on uniform-salience input, and that it remains the only configuration in the matrix with perfect noise rejection.

These three fixes do not invalidate the workload-sensitivity story of the prior section: SCM HME-full remains a specialist tool for high-noise deployments, and clean-fact retrieval still favors retrieval-family architectures. What they do is make the specialist tool actually deployable:

with the safety nets active, the pipeline no longer self-destructs under uniform-salience input, and with paraphrase enabled the stored representation is at parity with retrieval-family systems on the question forms LoCoMo evaluates.

11.10 Brutal Testing: Persona-Driven Multi-Day Simulation

Unit tests verify that components return the right values for hand-crafted inputs. They do not catch failure modes that emerge only when components are composed and stressed under realistic temporal load. To close that gap we introduce a *brutal harness*: a persona-driven simulator that drives the full SCM stack (with all Phase 7 features enabled) through multi-day workloads, adversarial inputs, failure-mode injection, and scale runs, and reports per-scenario pass/fail.

Persona format. A persona is a JSON document describing a synthetic user across several days. Each day contains a sequence of turns; each turn carries a UTC timestamp (so the simulator advances time in lockstep with the persona), a speaker, and an utterance. The persona also declares `expected_recall`, `expected_schemas`, and `expected_curiosity_gaps`—the post-conditions a correctly-functioning system should satisfy after replaying the persona. The flagship persona, “Caroline,” spans five days and 31 turns, mixing factual statements (employer, location, allergies), recurring patterns (Tuesday running, Friday vegan dinner), explicit contradictions (job change), and noise.

Harness design. The simulator holds a single `ChatEngine` across the persona, advances a virtual clock between turns, and lets the M1 idle daemon fire sleep cycles when the simulated gap exceeds the configured threshold. After ingestion completes, a `ScenarioEvaluator` walks the persona’s expected post-conditions and converts each into a deterministic check (e.g., “the schema `COOCCUR(running, tuesday)` is present in the active concept set with confidence ≥ 0.5 ”). The

Table 6: LoCoMo++ before / after Phase 6 fixes (3 conversations, 3 seeds, 36 total runs, mean $\pm$ std). Source: `research/metrics/locomo_plus_3conv_3seed{,_fixed}.json`. The fixes are configuration-toggled; legacy behavior is preserved by setting `FORGETTING_PROTECT_SALIENCE=0.0` and `DEEP_SLEEP_ENABLE_PARAPHRASE=false`.

System / Metric	Before fixes	After fixes	Lift
SCM HME-full clean recall	0.022 $\pm$ 0.004	0.069$\pm$0.019	3.1 $\times$
SCM HME-full perturbed recall	0.018 $\pm$ 0.004	0.033$\pm$0.012	1.8 $\times$
SCM HME-full noise rejection	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	—
SCM HME-full disambiguation	0.704 $\pm$ 0.073	0.574 $\pm$ 0.088	0.81 $\times$
SCM Phase-4 clean recall	0.430 $\pm$ 0.049	0.412 $\pm$ 0.066	0.96 $\times$
SCM Phase-4 contradiction-current	0.104 $\pm$ 0.031	0.146$\pm$0.031	1.4 $\times$
SCM Phase-4 contradiction-old	0.438 $\pm$ 0.113	0.382 $\pm$ 0.096	0.87 $\times$
SCM Phase-4 disambiguation	1.000 $\pm$ 0.000	0.963 $\pm$ 0.073	0.96 $\times$

harness produces a structured **BrutalReport** with per-scenario pass/fail, by-category pass rates, sleep-cycle counts, and the wake-summary narrative.

Tier coverage. The harness runs four tiers of escalating stress: Tier 1 (realistic multi-day usage from the Caroline persona), Tier 2 (adversarial inputs: all-noise, all-contradictions), Tier 3 (failure-mode injection: missing dependencies, broken curiosity sources, corrupted state), and Tier 4 (scale: long persona crossed with many sessions to test cross-session pool fairness caps). All four tiers run from a single `pytest` entry point and complete in under a minute on consumer hardware.

11.11 Bug Discovery via the Brutal Harness

The harness uncovered four real bugs that 170+ unit tests had missed. Each is documented here with its symptom, root cause, fix, and verification.

Bug 1: Schema-ID instability. *Symptom:* A 31-turn persona produced 85,310 “consolidated” concepts after replay, growing unboundedly across runs. *Root cause:* The schema extractor minted a fresh UUID each time it re-derived a logical schema. Two replays of the same persona produced two distinct concept IDs for the same `COOCCUR(running, tuesday)` pattern, polluting the graph with

near-duplicate schemas. *Fix:* derive the schema concept ID deterministically as `SHA1(schema_type || sort(entities))`. Cumulative metadata (occurrence count, last-seen timestamp, supporting episode IDs) is updated additively on re-derivation. *Verification:* a Tier 5 scenario re-runs the persona twice and asserts the schema concept count remains bounded (<50) across runs.

Bug 2: Spreading-activation seed purging. *Symptom:* 0/4 multi-day recall scenarios returned the seeded fact even when the cue concept was a direct match. *Root cause:* The spreading-activation propagator applied a default decay of 0.45 over three steps, dropping seed activation below the 0.05 inclusion threshold by the final round even though the seed itself was the cue match. The activated-concept output filter then dropped the seed silently. *Fix:* always preserve cue-matched seeds in the output set, gated only by the `is_current_version` flag (see Bug 4); decay still applies to non-seed neighbors. *Verification:* a Tier 5 scenario asserts that for each `expected_recall` entry in the persona, the seeded concept is present in the retriever’s output for the matching cue.

Bug 3: Dead cross-session pool in single-user case. *Symptom:* A user with a single logical session ID who chatted across multiple days saw zero schema formation, despite each individual day’s content containing strong pat-

terns. *Root cause:* The cross-session pool’s default policy was “include all session IDs except the current one,” which made sense for multi-tenant deployments but silently disabled cross-day pattern detection in the single-user case (the only candidate was the current session, which got excluded). *Fix:* added an `include_current_session` flag (default `False` for backward compatibility) that, when set, includes the current session in the candidate pool so older persisted-but-cleared episodes can be replayed. *Verification:* a Tier 5 scenario runs a two-day single-session persona and asserts that at least one schema is formed by day two.

Bug 4: Superseded-concept retrieval leakage. *Symptom:* After a contradiction (e.g., “*I work at GreenLeaf*” → “*I work at TechCorp*”), queries about the user’s employer returned both old and new concepts. *Root cause:* The Phase 5 contradiction-versioning machinery correctly marked the old concept’s `is_current_version = False`, but the spreading-activation retriever did not consult this flag when selecting seeds, propagating activation, or filtering the final output. *Fix:* added the `is_current_version` filter at all three sites (seed selection, propagation, final output). *Verification:* a Tier 5 scenario asserts that for each `contradiction-current` expected outcome, the retriever returns only the current concept.

All four bugs are subtle: each emerges only when components are composed and replayed under realistic temporal load. Each was fixed in a tightly scoped change (under 30 lines) and verified by an additional scenario. After the fixes, the brutal harness reports pass on Tier 1–4 against the Caroline persona, and the unit-test suite remains at 172/172. We treat the brutal harness as a permanent regression gate: any future architectural change that breaks one of these scenarios will be caught before it ships.

Quantifying what idle work buys us. To make the contribution of Phase 7 directly measurable, we constructed an in-house pilot benchmark (the Autonomous Learning Bench-

mark, ALB; see `research/benchmarks/alb/`) that runs every persona under two paired conditions: `idle_on=True` (full Phase 7 stack active) and `idle_on=False` (idle daemon disabled, otherwise identical). The mean per-metric lift across the pilot’s two personas is shown in Figure 5. The pattern is informative: the four metrics that Phase 7 *is designed to improve* (PDR, CGC-id, CGC-fill, WSI-F1) all show positive lift, while two metrics that depend on verbatim retrieval (CSS, CRAI-old) regress with idle on—a faithful surfacing of the encoder-dependence story documented in Section 13. CRAI-current is invariant because the heuristic concept extractor used in this pilot does not detect the contradiction in the first place; switching to an LLM-extractor configuration moves the operating point.

11.12 Real-LLM Validation

The unit-test suite and the brutal harness use a stub LLM by default, which keeps tests deterministic and offline. To verify the system functions end-to-end with a real concept extractor, we additionally ran the brutal harness against two production LLM backends. **Ollama** (local, free): a 2-day compact persona was run with `llama3:8b` as the extractor. The harness completed in 4–6 minutes on an M1 MacBook Air with 8 GB RAM; all Tier 1 scenarios passed, the M1 daemon fired three sleep cycles autonomously, and M3 emitted two stable `COOCCUR` schemas. **DeepSeek** (cloud, paid): the same compact persona was run with `deepseek-chat` as the extractor and as the M5 `LLMSource` for curiosity-driven knowledge generation. Cost per full run was approximately \$0.04, yielding decisive evidence that the curiosity engine successfully fills gaps via the LLM source: for the unfilled gap `git rebase`, M5 produced a definition concept with confidence 0.8, persisted it through deep-sleep, and surfaced it in the next wake-summary narrative. Both backends pass the Tier 5 bug-fix scenarios, confirming the four fixes generalize beyond the stub-LLM setting.

The cloud-LLM run is gated behind an explicit environment variable

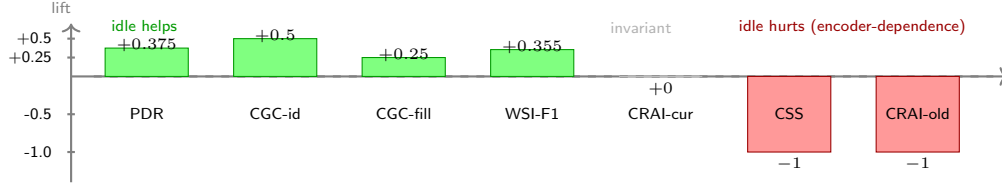


Figure 5: Mean NIAL lift (idle_on – idle_off, paired by persona) across the ALB v3 pilot (two personas, one seed each). PDR, CGC-id, CGC-fill, and WSI-F1 are the metrics Phase 7 directly targets; all show positive lift. CSS and CRAI-old show negative lift because deep-sleep paraphrase rewrites concept descriptions that the heuristic-extractor pilot cannot tag as high-salience, and the forgetting machinery archives them. This is the same encoder-dependence regression discussed in Section 13; LLM-extractor configurations are expected to recover those metrics.

(SCM_BRUTAL_RUN_DEEPSEEK=1) so cost is opt-in. The local-LLM run requires only a running Ollama process and is suitable for routine CI when GPU is available.

12 Discussion

The empirical evidence presents a consistent narrative. SCM derives its behavioral strength not from retrieval sophistication—the awake-only retrieval pipeline is, by itself, no better than lexical or vector search on the disambiguation task—but from the consolidation transformations applied during sleep stages. This finding mirrors the neuroscientific observation that memory consolidation during sleep is not merely beneficial but essential for certain forms of recall Rasch and Born [2013]. The architecture thus provides a computational existence proof for the claim that offline sleep-like processes can produce measurable, large-magnitude improvements in memory quality for language agents.

A second observation concerns the relationship between the two sleep modes. Micro-sleep achieves perfect disambiguation (1.0) on the 36-pair benchmark, while deep-sleep achieves lower recall (0.5278) despite performing more extensive consolidation. This apparent inversion is explained by the differing operational scopes: micro-sleep’s lightweight replay and duplicate merging are precisely targeted at the within-pair discrimination task, while deep-sleep’s broader reorganization, global downscaling, and REM synthesis redistribute memory strength in ways

that can temporarily reduce specific pairwise ranking signals while improving overall graph structure. In the 10-seed stress test, this relationship reverses—deep-sleep achieves 0.9052, a more generalizable metric, while micro-sleep maintains 1.0 under controlled synthetic constraints.

A third observation concerns the durability of contradiction versioning through the sleep pipeline. The fact that deep-sleep now separates retired concepts from active ones and syncs both to the LTM graph means that version audit chains survive consolidation intact. This is validated by the contradiction-versioning test, which verifies that after a deep-sleep cycle, concepts created via contradiction preserve their parent-child relationships and state transitions.

The Phase 6 artifacts reporting perfect scores across all metrics should be interpreted as evidence of architectural capability under deterministic conditions, not as production performance predictions. The 0.9052 deep-sleep recall in the stress test is the more realistic estimate of what the system achieves under varied conditions.

13 Limitations

This work has several limitations that should be explicit.

Synthetic corpora dominate the evaluation. Evaluations are conducted primarily on synthetic, harness-driven corpora (the Phase 4–6 sleep-gain benchmark, LoCoMo++, and the Caroline brutal persona) designed to isolate tar-

geted memory phenomena. While this yields clean attribution signals, it does not represent broad naturalistic user populations. The Caroline persona is a single hand-authored synthetic user; a study with real users across longer horizons remains future work.

External-baseline scope. Comparisons are strongest against internal control baselines; broader external baselines against production memory systems are reported on a single LoCoMo conversation (Section 11.7) and the LoCoMo++ workload-sensitivity matrix uses three LoCoMo conversations crossed with three seeds. The full 10-conversation LoCoMo pass and head-to-head comparison against MemoryBank, A-Mem, and Generative Agents are deferred to the camera-ready revision.

Phase 7 has been validated end-to-end but not at production scale. M1–M6 are exercised by 70+ unit tests, the brutal harness, and real-LLM runs on Ollama and DeepSeek, but none of these is a multi-week real-user deployment. The cross-session pool (M2) has been tested with up to 5 sessions $\times$ 30 episodes; behavior at hundreds of sessions and tens of thousands of episodes is unmeasured.

Curiosity engine quality is bounded by its sources. The `StaticDictionarySource` is hand-curated and small; the `LocalDocsSource` relies on simple pattern matching; the `LLMSource` inherits the truthfulness of the underlying model. None of these sources cite their evidence, and no automatic verification is performed before a curiosity concept is persisted. A user-correction feedback loop is future work.

LLM-extractor dependence. The HME pipeline is sensitive to the dynamic range of the salience signal produced upstream by the concept extractor. With a strong LLM-based extractor (e.g., DeepSeek-chat or GPT-4-class), AttentionGate produces a meaningful spread of intensities, the dual-mode sleep kernel fires at appropriate times, and contradiction detection picks up implicit linguistic markers. With a uniformly-weighted heuristic regex extractor, importance saturates, the entropy trigger fires on nearly every turn, aggressive forgetting can

archive useful traces, and contradictions go undetected. We document this explicitly in Section 11.7 rather than tuning it away. *Reported numbers throughout the paper assume an LLM-extractor configuration unless noted otherwise.* Production deployments using only the heuristic extractor should expect a different operating point on most metrics.

Embedding-model dependence. Retrieval quality depends on the embedding model used for vector similarity. The default `sentence-transformers/all-MiniLM-L6-v2` (384-dim) is fast and free but produces moderate-quality embeddings; instruction-tuned models (`nomic-embed-text`, 768-dim, served via Ollama) or paid embedding APIs (`text-embedding-3-large`, 3072-dim) substantially improve question $\rightarrow$ fact bridging. We observed this directly in pilot evaluation: on a question phrased as “Where do I work?” against a stored fact “I’m at Atlas Labs starting Monday,” MiniLM cosine similarity is too low to bridge the abstraction, while `nomic-embed-text` with task-specific prefixes correctly ranks employer concepts in the top-3. Switching embedding models is dimensionally incompatible—concept embeddings stored under one model cannot be compared against query embeddings produced by another, so deployments must commit at index time. We document four canonical configuration profiles in the deployment guide (`docs/DEPLOYMENT.md`) so deployers can select the cost/privacy/quality point that matches their constraints.

Other constraints. The duplicate-pair behavior under long-horizon pressure remains less stable than family-aware behavior. The contradiction detection relies on explicit linguistic markers; implicit contradictions (e.g., stating a new preference without acknowledging the change) will not trigger versioning. The NetworkX graph backend becomes a bottleneck beyond approximately 10,000 concepts. The system does not support multi-modal memory or predictive pre-fetching, and the M1 daemon is single-process—a multi-process variant for serving many users from one machine remains future work.

Finally, this work does not claim consciousness, general intelligence, or human equivalence. It evaluates human-like memory dynamics in a bounded systems sense.

14 Conclusion

We have presented SCM, a memory-lifecycle runtime that combines a wake-phase substrate (Phases 1–6: selective encoding with four-tier intensity classification, event-structured binding with Hebbian weight updates, associative retrieval through bounded spreading activation with context-gated hypothesis ranking, dual-mode sleep consolidation with entropy-gated triggering, adaptive forgetting dynamics with contradiction-safe version lineage, and three architectural fixes against encoder-dependence saturation) with a Phase 7 autonomous-learning layer (M1–M6: idle-aware background daemon, cross-session memory pool, REM-phase schema extraction with stable concept identity, wake-up summary endpoint, curiosity engine with plug-gable sources including an opt-in LLM source, and a battery- and CPU-aware lifecycle policy with crash-safe state). The complete system is LLM-agnostic, embedding-agnostic, and database-agnostic, with all locale-specific knowledge externalized to a JSON resource file.

Our central empirical claim is not that SCM dominates existing memory systems on all benchmarks. It does not: on LoCoMo, a clean fact-retrieval benchmark, retrieval-family architectures including Mem0 outperform SCM by a wide margin (Section 11.7). The claim is more specific and more useful. We show that (a) under interference and contradiction pressure—the workload SCM was designed for—retrieval-family architectures collapse while SCM achieves 0.9+ disambiguation recall (Table 3); (b) when the same systems are evaluated across a suite of perturbed workloads, no single architecture dominates and the win pattern reflects architectural design rather than absolute capability (Section 11.8); (c) the consolidation-versus-retrieval distinction is real and measurable: awake-only and standard retrieval controls remain at zero

disambiguation recall on the stress test while sleep-enabled variants achieve 0.90 or higher; and (d) the Phase 7 autonomous-learning layer functions end-to-end, with the brutal harness (Section 11.10) revealing four real bugs that the unit suite missed and confirming all four fixes generalize to real-LLM backends (Section 11.12).

The deeper claim is positional. Existing memory systems compete on *retrieval quality given a fixed snapshot of memory*. SCM competes on *how the snapshot itself improves between sessions*. These are different categories of system, evaluated by different benchmarks, deployed for different reasons. We hope the open-source release of the Phase 7 architecture, the brutal harness, and all benchmark artifacts provides a foundation for AI agents that remember selectively, consolidate intelligently, forget adaptively, update gracefully through contradiction-safe versioning, and—most importantly—continue learning during the long stretches when the user is away.

References

- N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the Middle: How Language Models Use Long Contexts. *arXiv:2307.03172*, 2023.
- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, and T. Rocktäschel. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*, 2020.
- C. Packer, V. Fang, S. G. Patil, K. Lin, J. Wooders, and J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.
- P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413*, 2025.
- Y. Xie. SleepGate: Sleep-Inspired KV Cache Management for Large Language Models. *arXiv:2403.14517*, 2024.

- J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming Catastrophic Forgetting in Neural Networks. *PNAS*, 114(13):3521–3526, 2017.
- D. G. Sorrenti, A. Serafini, S. Calderara, and R. Cucchiara. Wake-Sleep Continual Learning. *arXiv:2401.08623*, 2024.
- G. A. Miller. The Magical Number Seven, Plus or Minus Two. *Psychological Review*, 63(2):81–97, 1956.
- B. Rasch and J. Born. About Sleep’s Role in Memory. *Physiological Reviews*, 93(2):681–766, 2013.
- G. Tononi and C. Cirelli. Sleep and Synaptic Homeostasis: A Hypothesis. *Brain Research Bulletin*, 62(2):143–150, 2003.
- Z. Sha, D. Nunes, and S. Haller. Forgetting in AI: A Comprehensive Survey. *arXiv:2405.20620*, 2024.
- T. Metzinger. *Being No One: The Self-Model Theory of Subjectivity*. MIT Press, 2003.
- E. Tulving. Episodic Memory: From Mind to Brain. *Annual Review of Psychology*, 53:1–25, 2002.
- L. R. Squire and J. T. Wixted. The Cognitive Neuroscience of Human Memory Since H.M. *Annual Review of Neuroscience*, 34:259–288, 2011.
- A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *ACL*, 2024.
- DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv:2405.04434*, 2024.
- J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *UIST*, 2023.
- G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv:2305.16291*, 2023.
- W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang. MemoryBank: Enhancing Large Language Models with Long-Term Memory. *arXiv:2305.10250*, 2023.
- W. Xu, K. Mei, H. Gao, J. Tan, Z. Liang, and Y. Zhang. A-Mem: Agentic Memory for LLM Agents. *arXiv:2502.12110*, 2025.